

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



BÁO CÁO THỰC NGHIỆM

Học phần: An toàn & Bảo mật thông tin

**Chủ đề: Xây dựng chương trình mã hóa và giải mã
RSA (Sử dụng ngôn ngữ C++, Java)**

Giáo viên hướng dẫn : TS. Phạm Văn Hiệp

Nhóm sinh viên thực hiện :

- | | |
|---------------------|-------------------|
| 1. Nguyễn Đức Dũng | Mã SV: 2024606323 |
| 2. Nguyễn Trung Duy | Mã SV: 2024605900 |
| 3. Triệu Xuân Lập | Mã SV: 2024605284 |
| 4. Vũ Anh Phúc | Mã SV: 2024605001 |
| 5. Đoàn Đức Tiến | Mã SV: 2024606986 |

Mã Lớp học phần: 20252IT6001002

Nhóm: 3

Hà Nội - Năm 2026

MỤC LỤC

LỜI NÓI ĐẦU.....	7
CHƯƠNG 1: TỔNG QUAN.....	8
1.1. Tổng quan về An toàn bảo mật thông tin.....	8
1.1.1. Khái niệm an toàn bảo mật thông tin	8
1.1.2. Thách thức và giải pháp an toàn bảo mật thông tin.....	8
1.1.3. Vai trò của mã hóa công khai trong an toàn bảo mật thông tin.....	11
1.2. Các kiến thức cơ sở.....	12
1.2.1 Cơ sở về toán học	12
1.2.2. Cơ sở về thuật toán	14
1.2.3. Giới thiệu về ngôn ngữ lập trình sử dụng	15
1.3. Nội dung nghiên cứu	16
1.3.1. Lý do nghiên cứu	16
1.3.2. Nội dung nghiên cứu.....	17
CHƯƠNG 2: KẾT QUẢ NGHIÊN CỨU.....	18
2.1. Hệ mã hóa khóa công khai.....	18
2.2. Nghiên cứu, tìm hiểu về hệ mật mã Elgamal.....	21
2.2.1. Cấu trúc và nguyên lý hoạt động của thuật toán Elgamal.....	21
2.2.2. Phân tích ưu và nhược điểm của hệ mật mã Elgamal.....	22
2.2.3. Các ứng dụng thực tế của thuật toán Elmagal trong bảo mật dữ liệu	23
2.3. Thiết kế chương trình, cài đặt thuật toán.....	25
2.3.1. Thiết kế chương trình.....	25
2.3.2. Giới thiệu ngôn ngữ lập trình sử dụng để cài đặt thuật toán.....	28
2.3.3. Cài đặt thuật toán, giao diện chương trình.....	35
CHƯƠNG 3: KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM	54
3.1. Kiến thức kỹ năng đã học được trong quá trình thực hiện đề tài.....	54
3.2. Bài học kinh nghiệm	55
3.3 Đề xuất về tính khả thi của chủ đề nghiên cứu, những thuận lợi, khó khăn.	55
3.3.1. Tính khả thi của đề tài.....	55
3.3.2. Thuận lợi	56

3.3.3. Khó khăn	56
TÀI LIỆU THAM KHẢO.....	57

Bảng phân công công việc

Tuần	Người thực hiện	Nội dung công việc	Kết quả đạt được	Kiến nghị với giảng viên
1	Nguyễn Đức Dũng	Xây dựng kế hoạch thực hiện đề tài, phân công nhiệm vụ, lập đề cương báo cáo.	100%	
	Nguyễn Trung Duy	Viết phần tổng quan về an toàn và bảo mật thông tin.(Mục1.1)	100%	
	Vũ Anh Phúc	Viết phần giới thiệu về ngôn ngữ lập trình sử dụng.(Mục1.2.3)	100%	
	Triệu Xuân Lập	Viết phần cơ sở toán học. (Mục1.2.1)	100%	
	Đoàn Đức Tiến	Viết phần cơ sở thuật toán. (Mục1.2.2)	100%	
2	Nguyễn Đức Dũng	Viết phần lý do nghiên cứu, nội dung nghiên cứu. (Mục1.3)	100%	
	Nguyễn Trung Duy	Viết phần hệ mã khóa công khai. (Mục2.1)	100%	
	Vũ Anh Phúc	Viết phần cấu trúc và nguyên lý hoạt động của thuật toán Elgamal. (Mục2.2.1)	100%	
	Triệu Xuân Lập	Viết phần phân tích ưu và nhược điểm của hệ mật mã Elgamal. (Mục2.2.2)	100%	
	Đoàn Đức Tiến	Viết phần Các ứng dụng thực tế của thuật toán Elmagal trong	100%	

		bảo mật dữ liệu. (Mục2.2.3)		
3	Nguyễn Đức Dũng (Python)	Viết phần thiết kế chương trình, giới thiệu ngôn ngữ lập trình sử dụng để cài đặt thuật toán. (Mục2.3.1 và 2.3.2)	100%	
	Nguyễn Trung Duy (Python)	Cài đặt chức năng Tạo khóa và Sinh mã tự động Elgamal trên môi trường Python.	100%	
	Vũ Anh Phúc (Python)	Cài đặt chức năng Mã hóa và Lưu file Elgamal trên môi trường Python.	100%	
	Triệu Xuân Lập (C++)	Thiết kế giao diện UI C++.	100%	
	Đoàn Đức Tiến (C++)	Cài đặt chức năng Giải mã và Sinh khóa tự động trên C++.	100%	
4	Nguyễn Đức Dũng (Python)	Thiết kế giao diện UI Python tích hợp các chức năng thành chương trình hoàn chỉnh.	100%	
	Nguyễn Trung Duy (Python)	Kiểm thử và đánh giá kết quả trên Python, chụp ảnh minh họa. (Mục2.3.3.2)	100%	
	Vũ Anh Phúc (Python)	Viết phần cài đặt thuật toán. (Mục2.3.3.1)	100%	
	Triệu Xuân Lập (C++)	Thiết kế giao diện UI C++(tiếp tục).	100%	

	Đoàn Đức Tiến (C++)	Kiểm thử và đánh giá kết quả trên C++, chụp ảnh minh họa. (Mục 2.3.3.2)	100%	
5	Nguyễn Đức Dũng	Tổng hợp toàn bộ file Word. Format chuẩn chỉ theo quy định.	100%	
	Nguyễn Trung Duy	Viết Chương 3 - Mục 3.1: Kiến thức và kỹ năng đã học được.	100%	
	Vũ Anh Phúc	Viết Chương 3 - Mục 3.2: Bài học kinh nghiệm.	100%	
	Triệu Xuân Lập	Viết Chương 3 - Mục 3.3: Tính khả thi của đề tài	100%	
	Đoàn Đức Tiến	Viết Chương 3 - Mục 3.3: Thuận lợi và khó khăn.	100%	

LỜI NÓI ĐẦU

Trong kỷ nguyên số hóa, an toàn và bảo mật thông tin đã trở thành một yêu cầu cấp thiết đối với mọi hệ thống công nghệ. Các kỹ thuật mã hóa dữ liệu chính là chiếc chìa khóa cốt lõi để bảo vệ thông tin trước những nguy cơ xâm nhập ngày càng tinh vi. Nhận thức được tầm quan trọng đó, chúng em đã chọn đề tài: "Xây dựng chương trình mã hóa và giải mã ElGamal". Đề tài đi sâu vào nghiên cứu thuật toán mã hóa phi đối xứng ElGamal - một hệ mật mã nổi bật với độ an toàn cao dựa trên bài toán logarit rời rạc.

Việc thực hiện đề tài không chỉ giúp sinh viên nắm vững nền tảng lý thuyết mật mã học, mà còn tạo cơ hội để ứng dụng thực hành thông qua việc xây dựng chương trình bằng **ngôn ngữ C++ và Python**. Nếu như C++ mang lại ưu điểm tuyệt đối về tốc độ xử lý tính toán, thì Python lại hỗ trợ triển khai các thuật toán một cách linh hoạt và trực quan. Mục tiêu cuối cùng của đề tài là sự kết hợp hài hòa giữa kiến thức hàn lâm và kỹ năng thực chiến, góp phần nâng cao tư duy logic và năng lực lập trình của sinh viên IT.

CHƯƠNG 1: TỔNG QUAN

1.1. Tổng quan về an toàn bảo mật thông tin

1.1.1. *Khái niệm an toàn bảo mật thông tin*

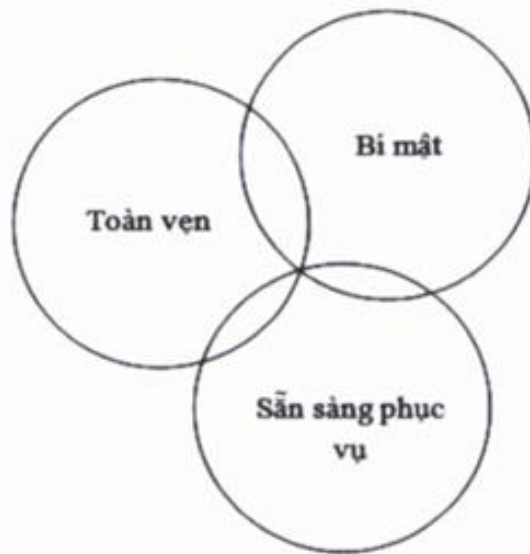
Trong xã hội bùng nổ thông tin, khi mà dữ liệu thông tin đóng vai trò tài sản giá trị chiến lược vô giá, quyết định sự sống còn và lợi thế cạnh tranh thành bại của mọi công tác nghiệp vụ từ các tổ chức, doanh nghiệp nhỏ đến các tập đoàn lớn xuyên quốc gia, các cơ quan an ninh, các tổ chức chính trị, xã hội cho đến các trường học, viện nghiên cứu thì vấn đề đảm bảo được an ninh thông tin là một vấn đề được đặt lên hàng đầu.

Theo lý thuyết an toàn thông tin hệ thống, thông tin được hiểu là tập hợp các dữ liệu, tin tức phản ánh về thế giới xung quanh chúng ta (các sự kiện, các cá nhân, các hiện tượng, các quá trình, các nhân tố và các mối liên hệ giữa chúng), được thể hiện trong dạng thức phù hợp cho việc truyền đi bởi những người này và tiếp nhận bởi những người kia và được sử dụng với mục đích thu nhận kiến thức (các tri thức) và đưa ra những quyết định. Những thông tin này được hình thành, tồn tại và vận động trong các hệ thống thông tin - viễn thông.

Hệ thống thông tin - viễn thông là tập hợp các thiết bị kỹ thuật và bảo đảm phần mềm, liên hệ với nhau bằng các kênh truyền và nhận thông tin. Từ các yếu tố ngăn cách nhau về vị trí địa lý, chúng liên kết chặt chẽ với nhau thành một thể thống nhất nhằm mục đích bảo đảm chu trình công nghệ xử lý thông tin (tìm kiếm, lưu trữ, bảo vệ, xử lý, hiện đính) và cung cấp cho người dùng kết quả của sự xử lý này ở dạng đòi hỏi (yêu cầu). Tóm lại, hệ thống thông tin - viễn thông bao gồm các mạng máy tính, các bảo đảm toán học (các phần mềm) và hệ thống liên lạc.

1.1.2. *Thách thức và giải pháp an toàn bảo mật thông tin*

Để đánh giá tính an toàn của một hệ thống, các chuyên gia an ninh mạng dựa trên bộ ba thuộc tính cốt lõi thường được gọi là tam giác bảo mật CIA. Sự xáo trộn hay mất mát bất kỳ thuộc tính nào trong bộ ba này đều dẫn đến những tổn thất nghiêm trọng cho tổ chức. Trên thực tế khó phân biệt ranh giới giữa chúng. Ba phạm trù này có những miền giao nhau. Dễ thấy rằng, có những thông tin mật dành riêng cho một đối tượng dùng mà việc đáp ứng tính bí mật đã bao hàm cả sự toàn vẹn và sẵn sàng phục vụ rồi. Có thể miêu tả quan hệ giữa ba tính chất cơ bản của thông tin trong sơ đồ sau:



Hình 1.1: Mối quan hệ giữa 3 tính chất cơ bản của thông tin

Đảm bảo tính bí mật (Confidentiality): có nghĩa là ngăn chặn/phát hiện/cản trở những truy nhập thông tin trái phép. Nói chung, tính bí mật được sử dụng để bảo vệ dữ liệu trong những môi trường bảo mật cao như các trung tâm quân sự hay kinh tế quan trọng, bảo vệ tính riêng tư của dữ liệu

Đảm bảo tính toàn vẹn (Integrity): có nghĩa là ngăn chặn/phát hiện/cản trở các sửa đổi thông tin trái phép

Đảm bảo tính sẵn sàng (Availability): có nghĩa là ngăn chặn/phát hiện/cản trở sự từ chối trái phép các truy nhập hợp pháp đến dịch vụ trong hệ thống.

Như vậy vấn đề an toàn thông tin có thể được hiểu là vấn đề đảm bảo ba thuộc tính cơ bản của thông tin là: tính toàn vẹn, tính bí mật và tính sẵn sàng. Ba thuộc tính này của thông tin có thể bị tác động và ảnh hưởng bởi các hình thức tấn công hệ thống thông tin mà ta quan tâm dưới đây.

Các loại tấn công:



Hình 1.2: Các tấn công bị động và chủ động

Tấn công bị động (Passive Attacks):

+ **Cơ chế hoạt động:** là kiểu tấn công chặn bắt thông tin như: nghe trộm và quan sát truyền tin mà không làm thay đổi trạng thái của tài nguyên hệ thống.

+ **Mục tiêu:** Thu thập nội dung thông điệp bí mật hoặc phân tích lưu lượng để dự đoán hành vi giao tiếp.

+ **Khả năng phát hiện và phòng ngừa:** Cực kỳ khó phát hiện vì không làm thay đổi cấu trúc dữ liệu. Biện pháp đối phó tối ưu là mã hóa ngăn ngừa ngay từ đầu.

Tấn công chủ động (Active Attacks):

+ **Cơ chế hoạt động:** Là các tấn công sửa đổi luồng dữ liệu hay tạo ra luồng dữ liệu giả và có thể được chia làm bốn loại nhỏ là: đóng giả (Masquerade), dừng lại (Replay), sửa đổi thông báo (Modification of messages), từ chối cung cấp dịch vụ (Denial of Service).

+ **Mục tiêu:** Giả mạo thực thể, phát lại thông điệp cũ, chỉnh sửa nội dung gói tin hoặc chặn đứng kênh truyền.

+ **Khả năng phát hiện và phòng ngừa:** Dễ phát hiện hơn thông qua các hệ thống giám sát và kiểm tra toàn vẹn. Trọng tâm là phát hiện sớm và khôi phục nhanh sau sự cố.

Chúng ta thấy rằng hai kiểu tấn công chủ động và bị động có những đặc trưng khác nhau. Kiểu tấn công bị động khó phát hiện nhưng có biện pháp để ngăn chặn thành công. Ngược lại, kiểu tấn công chủ động dễ phát hiện nhưng lại rất khó ngăn chặn tuyệt đối, nó cũng đòi hỏi phải bảo vệ vật lý đối với tất cả các phương tiện truyền thông ở mọi lúc, mọi nơi. Giải pháp để chống lại kiểu tấn công này là phát hiện chúng và khôi phục mạng sau khi mạng bị tấn công và thông tin bị trộm.

Để giảm thiểu các rủi ro trên, cần áp dụng đồng bộ nhiều giải pháp:

- Xây dựng chính sách và quy trình quản lý an toàn thông tin.
- Sử dụng tường lửa, hệ thống phát hiện và ngăn chặn xâm nhập.
- Áp dụng cơ chế xác thực và phân quyền truy cập.
- Sử dụng các thuật toán mật mã để bảo vệ dữ liệu.
- Sao lưu dữ liệu định kỳ và xây dựng phương án khôi phục sự cố.
- Nâng cao nhận thức bảo mật cho người dùng thông qua đào tạo và tuyên truyền.

Việc kết hợp giữa các giải pháp kỹ thuật và quản lý giúp nâng cao khả năng bảo vệ hệ thống thông tin trước các mối đe dọa ngày càng phức tạp.

1.1.3. Vai trò của mã hóa công khai trong an toàn bảo mật thông tin

Mã hóa công khai (Public Key Cryptography) – hay **mã hóa bất đối xứng** – là một bước tiến mang tính cách mạng, giải quyết được những hạn chế lớn của mã hóa đối xứng truyền thống. Cơ chế này sử dụng một cặp khóa toán học: **Khóa công khai (Public Key)** dùng để mã hóa và **Khóa bí mật (Private Key)** dùng để giải mã.

Vai trò của nó được thể hiện rõ rệt qua các khía cạnh sau:

- **Bảo mật thông tin không cần chia sẻ khóa trước:** Người gửi có thể dùng khóa công khai của người nhận (vốn được công bố rộng rãi) để mã hóa dữ liệu. Chỉ duy nhất người nhận sở hữu khóa bí mật tương ứng mới giải mã được. Điều này giúp loại bỏ hoàn toàn rủi ro bị lộ khóa trong quá trình phân phối trên môi trường Internet.
- **Nền tảng của Chữ ký số (Digital Signature):** Khi người gửi dùng khóa bí mật của mình để ký vào văn bản, người nhận dùng khóa công khai để xác thực. Quá trình này mang lại hai vai trò lớn:
 - **Xác thực danh tính (Authentication):** Chứng minh văn bản đó chắc chắn đến từ người sở hữu khóa bí mật chứ không phải kẻ giả mạo.

- **Chống chối bỏ (Non-repudiation):** Người gửi không thể phủ nhận việc mình đã ký hay gửi thông điệp đó, tạo cơ sở pháp lý cho các giao dịch điện tử.
- **Đảm bảo tính toàn vẹn:** Chữ ký số đi kèm mã hóa công khai giúp phát hiện ngay lập tức nếu dữ liệu bị chỉnh sửa dù chỉ một bit trên đường truyền.
- **Trái tim của các giao thức bảo mật hiện đại:** Mã hóa công khai là công nghệ cốt lõi thiết lập nên các kết nối an toàn mà chúng ta dùng hàng ngày như **HTTPS** (giao thức SSL/TLS khi lướt web, ngân hàng trực tuyến), **SSH** (quản trị mạng từ xa), và bảo mật trong công nghệ **Blockchain**.

1.2. Các kiến thức cơ sở

1.2.1. Cơ sở toán học

1.2.1.1. Lý thuyết số

- Ước chung lớn nhất và số nguyên tố bằng nhau

Nếu (d) là ước của cả (a) và (b) thì (d) được gọi là ước chung của (a) và (b).

Ước chung lớn nhất của (a) và (b) được ký hiệu là: $\gcd(a, b)$

Nếu:

$$\gcd(a, b) = 1$$

thì (a) và (b) được gọi là hai số nguyên tố cùng nhau.

- Số nguyên tố

Số nguyên tố là số nguyên dương lớn hơn 1 và chỉ có hai ước là 1 và chính nó.

Ví dụ: (2, 3, 5, 7, 11, 13, ...)

Việc sử dụng các số nguyên tố lớn giúp tăng độ khó của các bài toán toán học trong mật mã học, từ đó nâng cao tính bảo mật của hệ mật ElGamal.

1.2.1.2. Số học modulo

Số học modulo là nền tảng của nhiều hệ mật khóa công khai, đặc biệt là ElGamal.

a. Phép chia dư

Với $a, n \in \mathbb{Z}$, n là số nguyên dương. Biểu thức $a \bmod n$ (hay $a \% n$) đưa ra kết quả số dư r của phép chia a cho n , với $r \in \{0, \dots, n-1\}$ sao cho $n \nmid (a-r)$. r luôn là số nguyên không âm ngay cả khi a là số nguyên âm.

Ví dụ: $20 \bmod 7 = 6$ vì $20 = 2 \cdot 7 + 6$

b. Phép đồng dư

Hai số nguyên a và b được gọi là đồng dư modulo n nếu:

$$a \equiv b \bmod n \quad (2.1)$$

Điều này có nghĩa là a và b có cùng số dư khi chia cho n .

Phép đồng dư được sử dụng thường xuyên trong các phép toán mã hóa và giải mã của ElGamal.

c. Nghịch đảo modulo

Số nguyên y được gọi là nghịch đảo của $(x \bmod n)$ nếu:

$$x \cdot y \bmod n = 1 \quad (2.2)$$

Kí hiệu khác: x^{-1}

Nghịch đảo modulo được sử dụng trong quá trình giải mã bản mã trong hệ mật ElGamal.

1.2.1.3. Nhóm cyclic và căn nguyên thủy

Trong ElGamal, các phép toán được thực hiện trên nhóm cyclic modulo số nguyên tố.

Nếu một phần tử (g) có thể sinh ra toàn bộ các phần tử trong nhóm thì (g) được gọi là phần tử sinh hay căn nguyên thủy.

Ví dụ với $n = 13$, ta có :

$$(1)13 = \{1\}$$

$$(2)13 = \{1,2,4,8,3,6,12,11,9,5,10,7\} = \mathbb{Z}^*_{13}$$

$$(3)13 = \{1,3,9\}$$

Khái niệm này được sử dụng trực tiếp trong quá trình tạo khóa của ElGamal.

1.2.1.4. Bài toán logarit rời rạc

Độ an toàn của hệ mật ElGamal dựa trên bài toán logarit rời rạc.

Bài toán được phát biểu như sau:

Cho: $x \in g$, tìm x sao cho $g^x = x$;

Với các số nguyên tố lớn, hiện nay chưa tồn tại thuật toán hiệu quả để giải bài toán này trong thời gian đa thức. Chính vì vậy, việc suy ra khóa bí mật từ khóa công khai là rất khó, giúp đảm bảo tính an toàn cho hệ mật ElGamal.

1.2.2. Cơ sở thuật toán

1.2.2.1. Thuật toán bình phương và nhân

Thuật toán bình phương và nhân (Exponentiation by squaring) là một phương pháp tiếp cận hiệu quả giúp tính các số mũ lớn bằng cách chia nhỏ số mũ e 2 lần sau mỗi lần đệ quy, biểu diễn tổng quát:

$$x^e = \begin{cases} 1 & \text{nếu } e=0 \\ \left(x^{\frac{e}{2}}\right)^2 & \text{nếu } e \text{ chẵn} \\ \left(x^{\frac{e-1}{2}}\right)^2 \cdot x & \text{nếu } e \text{ lẻ} \end{cases} \quad (2.3)$$

Thuật toán bình phương và nhân được sử dụng để tính lũy thừa modulo với số mũ lớn một cách hiệu quả.

Thay vì thực hiện phép nhân lặp lại nhiều lần, thuật toán sẽ chia nhỏ số mũ và xử lý theo dạng nhị phân, từ đó giảm đáng kể thời gian tính toán.

1.2.2.2. Thuật toán Euclid mở rộng

Thuật toán Euclid mở rộng được sử dụng để tìm:

- Ước chung lớn nhất của hai số nguyên
- Nghịch đảo modulo

Thuật toán dựa trên biểu thức:

$$ax + by = \gcd(x, y) \quad (2.4)$$

Khi:

$$\gcd(x, n) = 1$$

thì có thể tìm được nghịch đảo modulo của (x).

Trong ElGamal, thuật toán này hỗ trợ quá trình giải mã dữ liệu.

1.2.2.3. Thuật toán Miller–Rabin

Trong quá trình sinh khóa ElGamal cần tạo ra các số nguyên tố lớn.

Thuật toán Miller–Rabin là thuật toán kiểm tra tính nguyên tố theo xác suất, có ưu điểm:

- Tốc độ nhanh
- Độ chính xác cao
- Phù hợp với các số nguyên lớn

Do đó, thuật toán này thường được sử dụng trong các hệ mật khóa công khai hiện đại để hỗ trợ quá trình sinh khóa.

1.2.3. Ngôn ngữ lập trình sử dụng

Để triển khai hệ mật ElGamal, nhóm sử dụng ngôn ngữ lập trình Java và C#.

1.2.3.1. C++

C++ là ngôn ngữ lập trình có hiệu năng cao, phù hợp với các bài toán yêu cầu tối ưu tốc độ xử lý và quản lý bộ nhớ.

Ngôn ngữ này hỗ trợ tốt cho việc triển khai các thuật toán mật mã nhờ khả năng xử lý phép toán ở mức thấp và thực hiện các phép tính trên số lớn với tốc độ nhanh.

Ngoài ra, C++ còn được sử dụng rộng rãi trong nghiên cứu và phát triển các hệ thống bảo mật.

1.2.3.2. Python

Python là ngôn ngữ lập trình có cú pháp đơn giản, dễ đọc và hỗ trợ nhiều thư viện mạnh phục vụ cho lĩnh vực mật mã học và xử lý dữ liệu.

Một số thư viện như:

- hashlib
- sympy
- pycryptodome

giúp việc xây dựng và kiểm thử các thuật toán mã hóa trở nên thuận tiện hơn.

Python phù hợp cho việc mô phỏng thuật toán, kiểm thử chương trình và nghiên cứu các hệ mật mã.

1.3. Nội dung nghiên cứu

1.3.1. Lý do nghiên cứu

- **Hệ mật mã elgamal:** Trong bối cảnh xã hội ngày càng phát triển, việc đảm bảo tính bảo mật của thông tin là một vấn đề quan trọng đối với mỗi cá nhân, tổ chức, quốc gia. Trong các giao thức mã hóa công khai, hệ mã Elgamal đã trở thành một điểm sáng khi đảm bảo tính bảo mật và xác thực cao, dựa trên cơ sở về bài toán logarit rời rạc (Discrete Logarithm Problem – DLP).

- **Hệ mã Elgama** được phát triển bởi Taher Elgamal, cho ra cơ chế mã hóa phức tạp, khó giải mã; phù hợp với các hệ thống cũng như chương trình cần bảo mật cao; gây khó khăn

cho kẻ tấn công khi sử dụng các kĩ thuật phá khóa truyền thống. Hệ mã ElGamal được ứng dụng vào nhiều lĩnh vực yêu cầu an toàn bảo mật hiện nay:

- **Ký kết hợp đồng điện tử:** Xác thực danh tính các bên tham gia, ngăn chặn việc giả mạo hoặc thay đổi nội dung các hợp đồng và thỏa thuận kinh tế.
- **Thương mại điện tử & Thanh toán trực tuyến:** Đóng vai trò là thuật toán chữ ký số để bảo vệ thông tin, chứng thực nguồn gốc dữ liệu khi mua sắm.
- **Tiền mã hóa (Cryptocurrency):** Biến thể chữ ký số ElGamal (như chữ ký Schnorr) được ứng dụng trong các chuỗi khối (blockchain) và ví điện tử để cấp phép giao dịch an toàn.
- **Mạng riêng ảo (VPN):** Ứng dụng trong giao thức trao đổi khóa Diffie–Hellman (nền tảng của ElGamal) để thiết lập kênh giao tiếp bí mật giữa người dùng và máy chủ.

Hệ mã hóa ElGamal áp dụng bài toán logarit rời rạc chính vì vậy độ an toàn của hệ mã hóa là rất lớn vì bài toán logarit rời rạc chưa có phương pháp hiệu quả để tính. Với 1 số p đủ lớn, thuật toán mã hóa ElGamal không có phương pháp thám mã hiệu quả.

Mặc dù ElGamal mang lại nhiều lợi ích, song vẫn tồn tại các khó khăn như yêu cầu bản mã kích thước lớn (nhiều gấp 2 lần thông điệp gốc), yêu cầu tính toán cao. Ngoài ra hiện nay về mặt lý thuyết các máy tính lượng tử với khả năng tính toán lớn có thể giải được bài toán logarit rời rạc vốn là cơ sở của ElGamal. Tuy nhiên, ElGamal vẫn là một hệ mã công khai mạnh mẽ.

- **Lý do nghiên cứu:** Nhận thấy được những tiềm năng của hệ mã ElGamal trong bối cảnh hiện nay, đề tài “Xây dựng chương trình mã hóa và giải mã ElGamal” mang lại những kiến thức về các giao thức mã hóa công khai, từ đó nắm bắt được nguyên lý và khả năng ứng dụng của các hệ mã hiện đại. Đề tài này tạo nền tảng cho các dự án nghiên cứu bảo mật nói riêng và các giải pháp kháng lượng tử trong tương lai nói chung. Hơn nữa, kiến thức về ElGamal và kỹ năng lập trình với Python và C++ là một lợi thế quan trọng cho sinh viên, giúp cạnh tranh trong thị trường công việc đòi hỏi chuyên môn cao trong các lĩnh vực an ninh mạng, blockchain và bảo mật truyền thông.

1.3.2. Nội dung nghiên cứu

Các nội dung nghiên cứu gồm:

- Tìm hiểu nguyên lý hoạt động của hệ mã ElGamal, bao gồm các bước tạo khóa (Key Generation), mã hóa (Encryption) và giải mã (Decryption).
- Đánh giá được ưu điểm và nhược điểm về hiệu năng và độ an toàn của hệ mật Elgamal.
- Thực hiện phương pháp thực nghiệm cài đặt chương trình mô phỏng mã hóa và giải mã ElGamal sử dụng ngôn ngữ Python và C++.

Nội dung nghiên cứu chi tiết sẽ được trình bày ở các chương sau của báo cáo.

CHƯƠNG 2: KẾT QUẢ NGHIÊN CỨU

2.1. Nghiên cứu, tìm hiểu hệ mã hóa khóa công khai.

2.1.1. Khái niệm và nguyên lý hoạt động của mã hóa khóa công khai.

2.1.1.1. Khái niệm

Hệ mã hóa khóa công khai là một hệ mật mã sử dụng một cặp khóa gồm khóa công khai và khóa bí mật, có liên hệ toán học với nhau. Trong đó, khóa công khai được công bố rộng rãi để mã hóa thông tin, còn khóa bí mật được giữ kín và chỉ người sở hữu mới có thể sử dụng để giải mã.

Hệ mã hóa khóa công khai cho phép các bên trao đổi thông tin một cách an toàn mà không cần phải chia sẻ khóa bí mật trước đó, nhờ đó giải quyết được bài toán phân phối khóa trong các hệ thống mã hóa truyền thống.

Một yêu cầu quan trọng của hệ thống là không thể suy ra khóa bí mật từ khóa công khai trong thời gian khả thi về mặt tính toán, nhằm đảm bảo tính bảo mật.

Trong thực tế, các hệ mã hóa khóa công khai thường được triển khai theo mô hình mã hóa lai (hybrid encryption), trong đó thuật toán khóa công khai được sử dụng để mã hóa một khóa phiên đối xứng, giúp tăng hiệu quả khi truyền tải dữ liệu có kích thước lớn.

2.1.1.2. Nguyên lý hoạt động

Hệ mã hóa khóa công khai hoạt động dựa trên nguyên tắc sử dụng hai khóa riêng biệt:

- Khóa công khai (Public Key): dùng để mã hóa dữ liệu.
- Khóa bí mật (Private Key): dùng để giải mã dữ liệu.

Hai khóa này có liên hệ toán học chặt chẽ với nhau, nhưng việc suy ngược từ khóa công khai về khóa bí mật là không khả thi trong thời gian hợp lý, nhờ vào tính chất một chiều của các bài toán toán học khó như phân tích số nguyên tố lớn (RSA), logarit rời rạc (ElGamal), hoặc đường cong elliptic (ECC).

Các bước hoạt động cơ bản:

- Tạo khóa (Key Generation): Hệ thống sinh ra một cặp khóa: khóa công khai và khóa bí mật. Khóa công khai sẽ được phân phối tự do, còn khóa bí mật được giữ kín bởi chủ sở hữu.
- Mã hóa (Encryption): Người gửi sử dụng khóa công khai của người nhận để mã hóa thông điệp cần truyền đi. Do đó, chỉ người nhận mới có thể giải mã thông tin.
- Giải mã (Decryption): Người nhận dùng khóa bí mật của mình để giải mã bản mã, từ đó khôi phục lại thông điệp ban đầu.

2.1.2. Các phương pháp mã hóa phổ biến hiện nay

Các phương pháp mã hóa hiện đại được chia thành hai nhóm chính: mã hóa đối xứng và mã hóa bất đối xứng, ngoài ra còn có phương pháp mã hóa lai, kết hợp ưu điểm của cả hai nhóm trên.

2.1.2.1. Mã hóa đối xứng (Symmetric Encryption)

Mã hóa đối xứng là phương pháp sử dụng cùng một khóa bí mật để thực hiện cả hai quá trình: mã hóa và giải mã dữ liệu. Do tính chất đơn giản và tốc độ xử lý cao, phương

pháp này thường được sử dụng trong các hệ thống cần xử lý lượng lớn dữ liệu trong thời gian ngắn.

Các thuật toán tiêu biểu:

- AES (Advanced Encryption Standard): là chuẩn mã hóa được sử dụng rộng rãi nhất hiện nay, được công nhận bởi chính phủ Hoa Kỳ và nhiều tổ chức quốc tế nhờ độ an toàn và hiệu suất cao.
- DES và 3DES: là các chuẩn mã hóa cũ, từng được sử dụng rộng rãi nhưng hiện nay không còn được khuyến nghị do độ dài khóa ngắn và dễ bị tấn công.
- ChaCha20: là thuật toán hiện đại có tốc độ cao và hiệu quả trên các thiết bị di động.
- Blowfish, Twofish: là các thuật toán thay thế DES, có khả năng mã hóa nhanh và bảo mật tốt

2.1.2.2. Mã hóa bất đối xứng (Asymmetric Encryption)

Khác với mã hóa đối xứng, mã hóa bất đối xứng sử dụng hai khóa khác nhau: khóa công khai để mã hóa và khóa bí mật để giải mã. Phương pháp này giải quyết được vấn đề phân phối khóa và thường được sử dụng trong các hệ thống truyền thông an toàn.

Các thuật toán tiêu biểu:

- RSA (Rivest–Shamir–Adleman): là một trong những thuật toán mã hóa bất đối xứng phổ biến nhất, được ứng dụng trong giao thức bảo mật Internet (SSL/TLS), xác thực và chữ ký số.
- ElGamal: dựa trên bài toán logarit rời rạc, được sử dụng trong các hệ thống mã hóa và chữ ký số.
- ECC (Elliptic Curve Cryptography): sử dụng toán học đường cong elliptic để tạo cặp khóa với kích thước nhỏ nhưng độ an toàn cao, phù hợp với thiết bị có tài nguyên hạn chế như IoT.
- DSA/ECDSA (Digital Signature Algorithm): là các thuật toán dùng để tạo và xác minh chữ ký số, được chuẩn hóa bởi NIST.

2.1.2.3. Mã hóa lai (Hybrid Encryption)

Do mã hóa bất đối xứng có hiệu suất thấp khi xử lý dữ liệu lớn, các hệ thống thực tế thường sử dụng phương pháp mã hóa lai, trong đó:

- Mã hóa bất đối xứng được dùng để trao đổi khóa phiên (session key).
- Mã hóa đối xứng được dùng để mã hóa nội dung chính của dữ liệu.

Phương pháp mã hóa lai được ứng dụng rộng rãi trong các giao thức bảo mật như TLS/SSL, PGP, VPN, và các hệ thống truyền tin bảo mật khác.

Việc lựa chọn phương pháp mã hóa phụ thuộc vào mục tiêu bảo mật, hiệu năng hệ thống và bối cảnh ứng dụng cụ thể. Trong các hệ thống hiện đại, kết hợp linh hoạt nhiều kỹ thuật mã hóa là hướng tiếp cận phổ biến nhằm tối ưu cả tính bảo mật lẫn hiệu suất.

2.1.3. Vai trò của mã hóa khóa công khai trong bảo mật thông tin.

Mã hóa khóa công khai (Public-Key Encryption) đóng vai trò đặc biệt quan trọng trong lĩnh vực bảo mật thông tin hiện đại. Nhờ đặc điểm sử dụng hai khóa khác nhau – một khóa công khai để mã hóa và một khóa bí mật để giải mã – hệ thống này đã mở ra nhiều ứng dụng mới giúp giải quyết các vấn đề cốt lõi trong việc bảo vệ dữ liệu và đảm bảo an toàn truyền thông.

- Giải quyết vấn đề phân phối khóa: Trong các hệ mã hóa đối xứng, việc chia sẻ khóa bí mật giữa các bên là một thách thức lớn. Nếu kẻ tấn công chặn được khóa trong quá trình phân phối, toàn bộ hệ thống bảo mật sẽ bị vô hiệu hóa. Mã hóa khóa công khai loại bỏ rủi ro này bằng cách cho phép người gửi sử dụng khóa công khai đã được công bố rộng rãi, trong khi người nhận giữ bí mật khóa riêng tư để giải mã. Điều này giúp đảm bảo an toàn cho quá trình thiết lập kênh liên lạc.
- Tăng cường bảo mật trong môi trường mạng mở: Trong các hệ thống truyền thông qua Internet – vốn là môi trường công cộng và dễ bị tấn công – mã hóa khóa công khai tạo điều kiện thiết lập các kênh truyền tin an toàn mà không cần gặp mặt trực tiếp để trao đổi khóa. Đây là nền tảng của các giao thức như SSL/TLS dùng trong HTTPS, giúp bảo vệ dữ liệu khi duyệt web, thanh toán trực tuyến hoặc truy cập tài khoản cá nhân.
- Hỗ trợ xác thực và chữ ký số: Mã hóa khóa công khai còn được sử dụng để xác thực danh tính và tạo chữ ký số. Ứng dụng này cực kỳ quan trọng trong thương mại điện tử, trao đổi tài liệu số, hệ thống quản trị doanh nghiệp và các giao dịch chính phủ điện tử.
- Tạo nền tảng cho hệ thống bảo mật hiện đại: Nhiều công nghệ và tiêu chuẩn bảo mật hiện đại đều dựa trên mã hóa khóa công khai, như: Xác thực người dùng (qua chứng thư số, USB token, smartcard); Trao đổi khóa an toàn (trong mã hóa lai – hybrid encryption); Mã hóa email (PGP, S/MIME); Cơ sở hạ tầng khóa công khai (PKI) – quản lý và xác thực khóa công khai thông qua bên thứ ba tin cậy.

2.2. Nghiên cứu, tìm hiểu về hệ mật mã Elgamal

2.2.1. Cấu trúc và nguyên lý hoạt động của thuật toán Elgamal

2.2.1.1. Cấu trúc của thuật toán ElGamal

- Thuật toán ElGamal gồm 3 bước chính:
- + Tạo khóa
- + Mã hóa
- + Giải mã

2.2.1.2. Nguyên lý hoạt động

❖ Giai đoạn tạo khóa:

- Chọn số nguyên tố q đủ lớn
- + q cần đủ lớn để đảm bảo độ an toàn mật mã
- Chọn a là một căn nguyên thủy của q
- + $a < q$
- + a và q là hai số nguyên tố cùng nhau, tức $\gcd(a, q) = 1$
- + $\text{Ord}_{q(a)} = \varphi(q) = q - 1$ (tức a sinh ra toàn bộ phần tử trong Z_q)
- Chọn $X_A < q - 1$
- + Đây là khóa riêng của người nhận
- Tính $Y_A = a^{X_A} \bmod q$
- + Đây là khóa công khai, dùng cho người gửi để mã hóa
- Kết quả xác định khóa:
- + Khóa công khai: $K_P = \{q, a, Y_A\}$
- + Khóa bí mật: $K_S = \{X_A\}$

❖ Giai đoạn mã hóa:

- Chọn thông điệp ban đầu M , sao cho $0 < M < q$
- Chọn số ngẫu nhiên $k < q$
- k phải được chọn ngẫu nhiên và khác nhau mỗi lần mã hóa để đảm bảo bảo mật
- Tính khóa tạm thời: $K = Y_A^k \bmod q$
- Tính thành phần bản mã:
- + Thành phần thứ nhất: $C_1 = a^k \bmod q$
- + Thành phần thứ hai: $C_2 = K \cdot M \bmod q$
- Bản mã: (C_1, C_2)

Lưu ý: sau khi gửi bản mã đi thì số ngẫu nhiên k sẽ được hủy

❖ Giai đoạn giải mã:

- Bản mã (C_1, C_2)
- Tính khóa tạm thời: $K = C_1^{X_A} \bmod q$
- Tìm bản rõ: $M = C_2 \cdot K^{-1} \bmod q$

2.2.2. Ưu và nhược điểm của hệ mật mã ElGamal:

Ưu điểm:

- **Bảo mật cao chống lại các cuộc tấn công giả mạo:** Các ngôn ngữ hỗ trợ các thuật toán mã hóa và kỹ thuật ký số mạnh mẽ, giảm nguy cơ giả mạo và xâm nhập trái phép.
- **Tính ngẫu nhiên trong mã hóa:** Mỗi lần mã hóa một thông điệp, ElGamal tạo ra bản mã khác nhau nhờ việc chọn số ngẫu nhiên k . Điều này giúp chống lại các cuộc tấn công dò (brute-force) và phân tích bản mã.
- **Tính linh hoạt trong ứng dụng:** ElGamal không chỉ được sử dụng cho mã hóa mà còn có thể được điều chỉnh để tạo chữ ký số, cung cấp khả năng xác thực và không thể chối bỏ. Tính linh hoạt này làm cho ElGamal phù hợp với nhiều kịch bản bảo mật, từ mã hóa email đến bảo vệ giao dịch tài chính.

Nhược điểm:

- **Kích thước bản mã lớn:** Bản mã của ElGamal bao gồm cặp (C_1, C_2) . Điều này dẫn đến kích thước bản mã gấp đôi so với thông điệp gốc, gây khó khăn trong các ứng dụng có hạn chế về băng thông, như thiết bị IoT hoặc mạng tốc độ thấp.
- **Khó khăn trong quản lý số ngẫu nhiên:** Quá trình mã hóa của ElGamal yêu cầu chọn một số ngẫu nhiên k . Số k phải được tạo ngẫu nhiên và không được tái sử dụng cho các thông điệp khác, nếu không sẽ làm giảm độ an toàn của hệ mã. Việc tạo số ngẫu nhiên chất lượng cao đòi hỏi thuật toán tạo số ngẫu nhiên mạnh, tăng độ phức tạp của triển khai.
- **Hiệu năng tính toán thấp:** ElGamal yêu cầu thực hiện các phép toán modulo lớn, bao gồm lũy thừa modulo và tính nghịch đảo modulo. Những phép toán này tốn kém tài nguyên tính toán, đặc biệt khi q lớn. So với RSA, ElGamal thường có tốc độ mã hóa và giải mã chậm hơn, đặc biệt trong các hệ thống yêu cầu xử lý thời gian thực.

2.2.3. Các ứng dụng thực tế của thuật toán Elmagal trong bảo mật dữ liệu

Ví dụ hệ mã Elgamal trong thực tế:

- **Ký kết hợp đồng điện tử:** Trong ký kết hợp đồng điện tử, Elgamal giải quyết được 3 vấn đề lớn:
 - + **Xác thực danh tính (Authentication):** Đảm bảo hợp đồng được ký chính xác bởi người sở hữu khóa bí mật, không thể giả mạo chữ ký.
 - + **Toàn vẹn dữ liệu (Integrity):** Đảm bảo nội dung hợp đồng (điều khoản, số tiền, ngày tháng) không bị chỉnh sửa dù chỉ một ký tự sau khi ký.

- + **Chống chối bỏ (Non-repudiation):** Một khi đã ký, người ký không thể phủ nhận "Tôi chưa từng ký hợp đồng này", vì chữ ký đó là duy nhất và gắn liền với khóa bí mật của họ.
- **Thương mại điện tử & Thanh toán trực tuyến:** Trong thương mại điện tử & thanh toán trực tuyến, ElGamal bảo vệ các thông tin nhạy cảm như số thẻ tín dụng, mã CVV và thông tin ví điện tử khi truyền qua internet.
 - + **Bảo mật đường truyền:** Trình duyệt của khách hàng dùng khóa công khai của cổng thanh toán để mã hóa thông tin thẻ.
 - + **Chống nghe lén:** Dữ liệu trên đường truyền hóa thành các chuỗi số vô nghĩa. Chỉ có máy chủ cổng thanh toán giữ khóa bí mật mới giải mã được.
 - + **Cơ chế mã hóa ngẫu nhiên:** Điểm đặc biệt của ElGamal là tính ngẫu nhiên (Probabilistic Encryption). Nếu khách hàng mua 2 đơn hàng có cùng số tiền và cùng số thẻ, hệ thống sẽ tạo ra 2 bản mã hoàn toàn khác nhau. Điều này ngăn chặn hacker đoán nội dung dựa trên quy luật lặp lại.
 - + **Xác thực đơn hàng:** Khi bạn nhấn "Đặt mua", hệ thống tạo ra một chữ ký số ElGamal gắn liền với chi tiết đơn hàng (mã sản phẩm, số tiền, địa chỉ giao). Người bán không thể tự ý sửa đổi số lượng hay giá tiền sau khi bạn đã ký.
 - + **Chống chối bỏ (Non-repudiation):** Một khi lệnh thanh toán đã được ký bằng khóa bí mật của người dùng, họ không thể khiêu nại rằng "tôi không bấm mua" (trừ khi bị lộ khóa). Điều này bảo vệ quyền lợi hợp pháp cho các sản phẩm thương mại điện tử.
 - + **Chứng thư số SSL/TLS:** ElGamal được tích hợp trong các giao thức bảo mật để thiết lập kênh truyền an toàn (HTTPS) giữa khách hàng và website thương mại điện tử.
- **Tiền mã hóa (Cryptocurrency):** Biến thể chữ ký số ElGamal (như chữ ký Schnorr) được ứng dụng trong các chuỗi khối (blockchain) và ví điện tử để cấp phép giao dịch an toàn.
 - + **Tạo địa chỉ ví (Khóa công khai):** Khi tạo ví Crypto, hệ thống dùng nguyên lý bài toán logarit rời rạc nhằm tạo khóa bí mật (Private key) từ đó suy ra khóa công khai (Public key). ElGamal đảm bảo tiến trình này được diễn ra một chiều: có khóa bí mật thì suy ra được địa chỉ của ví, nhưng ngược lại thì không.

- + **Ký xác nhận giao dịch (Chữ ký số):** Khi phải chuyển tiền (ví dụ: chuyển 1 BTC), ta phải dùng khóa bí mật để "ký" vào lệnh chuyển tiền đó. Chữ ký này (gồm bộ hai số giống như cặp (r, s) của ElGamal) nhằm mục đích xác minh ai là chủ sở hữu hợp pháp của số tiền mà không cần tiết lộ khóa bí mật cho mạng lưới mạng.
- **Mạng riêng ảo (VPN):** Ứng dụng trong giao thức trao đổi khóa Diffie–Hellman (nền tảng của ElGamal) để thiết lập kênh giao tiếp bí mật giữa người dùng và máy chủ. Nó và các biến thể của nó được ứng dụng vào giai đoạn quan trọng nhất: Thiết lập kênh truyền an toàn (Key Exchange) và Xác thực danh tính (Authentication).
 - + **Chữ ký số:** Các biến thể Elgamal được tích hợp vào **Chứng chỉ số (Digital Certificates)** của máy chủ VPN.
 - + **Xác thực máy chủ/người dùng:** Khi kết nối VPN, máy chủ sẽ gửi chứng chỉ số chứa chữ ký Elgamal/biến thể của Elgamal của nó cho thiết bị của bạn. Thiết bị của bạn sau đó dùng khóa công khai nhằm xác thực. Nếu xác thực hợp lệ, kết nối mạng riêng ảo mới được thiết lập

2.3 Thiết kế chương trình, cài đặt thuật toán.

2.3.1. Thiết kế kịch bản chương trình

Chương trình được xây dựng nhằm mô phỏng quá trình hoạt động của hệ mật mã ElGamal. Hệ thống hỗ trợ các chức năng chính gồm sinh khóa, mã hóa thông điệp, giải mã bản mã và quản lý tệp dữ liệu. Các kịch bản sau mô tả quá trình tương tác giữa người dùng và hệ thống trong từng chức năng của chương trình.

Kịch bản 1: Sinh khóa ElGamal

Thành phần	Mô tả
Tác nhân	Người dùng
Chức năng	Sinh khóa ElGamal
Dữ liệu vào	Các tham số p và g
Xử lý	1. Người dùng nhập các tham số p và g . 2. Hệ thống kiểm tra p có phải là số nguyên tố hay không.

	3. Hệ thống kiểm tra g có phải là phần tử sinh trong nhóm modulo p hay không. 4. Hệ thống sinh ngẫu nhiên khóa bí mật x thỏa mãn điều kiện: $1 \leq x \leq p - 2$. 5. Hệ thống tính khóa công khai $y = g^x \bmod p$. 6. Hệ thống tạo cặp khóa ElGamal gồm khóa công khai và khóa bí mật.
Dữ liệu ra	Khóa công khai (p, g, y) và khóa bí mật x
Thông báo	Sinh khóa thành công hoặc thông báo lỗi khi dữ liệu đầu vào không hợp lệ

Kịch bản 2: Mã hóa thông điệp

Thành phần	Mô tả
Tác nhân	Người dùng
Chức năng	Mã hóa thông điệp
Dữ liệu vào	Thông điệp m và khóa công khai (p, g, y)
Xử lý	1. Người dùng nhập thông điệp cần mã hóa hoặc tải thông điệp từ tệp dữ liệu. 2. Hệ thống kiểm tra tính hợp lệ của thông điệp đầu vào. 3. Hệ thống sinh ngẫu nhiên số k thỏa mãn điều kiện: $1 \leq k \leq p - 2$. 4. Hệ thống tính $C_1 = g^k \bmod p$. 5. Hệ thống tính $C_2 = m \times y^k \bmod p$. 6. Hệ thống tạo bản mã dưới dạng cặp giá trị (C_1, C_2) .
Dữ liệu ra	Bản mã (C_1, C_2)
Thông báo	Mã hóa thành công hoặc thông báo lỗi khi dữ liệu đầu vào không hợp lệ

Kịch bản 3: Giải mã bản mã

Thành phần	Mô tả
Tác nhân	Người dùng
Chức năng	Giải mã bản mã

Dữ liệu vào	Bản mã (C_1, C_2) và khóa bí mật x
Xử lý	1. Người dùng nhập bản mã hoặc tải bản mã từ tệp dữ liệu. 2. Người dùng nhập khóa bí mật x. 3. Hệ thống kiểm tra tính hợp lệ của bản mã và khóa bí mật. 4. Hệ thống tính $s = C_1^x \bmod p$. 5. Hệ thống tính nghịch đảo modulo của s là $s^{-1} \bmod p$. 6. Hệ thống khôi phục thông điệp gốc theo công thức: $m = C_2 \times s^{-1} \bmod p.$ 7. Hệ thống hiển thị kết quả giải mã.
Dữ liệu ra	Thông điệp gốc m
Thông báo	Giải mã thành công hoặc thông báo lỗi khi bản mã hoặc khóa bí mật không hợp lệ

Kịch bản 4: Quản lý tệp dữ liệu

Thành phần	Mô tả
Tác nhân	Lưu và tải khóa, bản mã
Chức năng	Lưu và tải khóa, bản mã
Dữ liệu vào	Tệp chứa khóa công khai, khóa bí mật hoặc bản mã
Xử lý	1. Người dùng lựa chọn chức năng lưu hoặc tải dữ liệu. 2. Hệ thống mở hộp thoại lựa chọn tệp. 3. Hệ thống thực hiện đọc dữ liệu từ tệp hoặc ghi dữ liệu ra tệp. 4. Hệ thống kiểm tra định dạng dữ liệu trước khi xử lý. 5. Hệ thống cập nhật dữ liệu lên giao diện chương trình.
Dữ liệu ra	Nội dung khóa hoặc bản mã được lưu hoặc tải thành công
Thông báo	Lưu dữ liệu thành công, tải dữ liệu thành công hoặc thông báo lỗi khi thao tác thất bại

Luồng hoạt động tổng quát của chương trình

Bước	Hoạt động
1	Bắt đầu chương trình
2	Sinh khóa ElGamal

3	Nhập thông điệp
4	Mã hóa thông điệp
5	Lưu bản mã
6	Tải bản mã và khóa bí mật
7	Giải mã bản mã
8	Hiển thị kết quả
9	Kết thúc chương trình

2.3.2. Giới thiệu ngôn ngữ lập trình sử dụng để cài đặt thuật toán

2.3.2.1. Ngôn ngữ lập trình C++

a. Ngôn ngữ C++

C++ là ngôn ngữ lập trình bậc cao, đa mục đích, được phát triển bởi Bjarne Stroustrup tại Bell Labs vào năm 1985. C++ được xây dựng dựa trên ngôn ngữ C và bổ sung các đặc tính lập trình hướng đối tượng, giúp tăng khả năng tái sử dụng và quản lý chương trình.

C++ là một trong những ngôn ngữ lập trình phổ biến nhất hiện nay, được sử dụng rộng rãi trong phát triển phần mềm hệ thống, ứng dụng desktop, trò chơi, hệ điều hành, hệ thống nhúng và các ứng dụng yêu cầu hiệu năng cao.

b. Các đặc điểm chính của C++

Hiệu năng cao

- Mã nguồn C++ được biên dịch trực tiếp thành mã máy.
- Tốc độ thực thi nhanh và sử dụng tài nguyên hệ thống hiệu quả.
- Phù hợp với các ứng dụng yêu cầu xử lý thời gian thực hoặc khối lượng dữ liệu lớn.

Hướng đối tượng

- Hỗ trợ đầy đủ các tính chất của OOP:
 - Đóng gói (Encapsulation)
 - Kế thừa (Inheritance)
 - Đa hình (Polymorphism)
 - Trừu tượng (Abstraction)
- Giúp chương trình dễ bảo trì, mở rộng và tái sử dụng mã nguồn.

Quản lý bộ nhớ linh hoạt

- Cho phép cấp phát và giải phóng bộ nhớ động thông qua con trỏ (pointer).
- Hỗ trợ kiểm soát trực tiếp tài nguyên hệ thống.
- Tăng hiệu suất nhưng đòi hỏi lập trình viên quản lý bộ nhớ cẩn thận.

Đa nền tảng

- Có thể biên dịch và chạy trên nhiều hệ điều hành:
 - Windows
 - Linux
 - macOS
 - Unix
- Hỗ trợ nhiều trình biên dịch như:
 - GCC
 - Clang
 - Microsoft Visual C++

Thư viện chuẩn phong phú

- Cung cấp nhiều cấu trúc dữ liệu:
 - Vector
 - List
 - Queue
 - Stack
 - Map
 - Set
- Cung cấp các thuật toán:
 - Sort
 - Search
 - Binary Search
 - Heap
 - Graph processing

Hỗ trợ lập trình song song (Multithreading)

- Thư viện <thread> hỗ trợ đa luồng.
- Giúp tăng tốc độ xử lý và tận dụng tài nguyên CPU đa nhân.

Khả năng tương thích với C

- Có thể sử dụng trực tiếp nhiều thư viện được viết bằng C.
- Thuận lợi trong phát triển các hệ thống lớn và phần mềm hệ thống.

c. C++ trong an toàn và bảo mật thông tin

C++ là một trong những ngôn ngữ được sử dụng rộng rãi trong lĩnh vực an toàn thông tin nhờ khả năng truy cập trực tiếp vào tài nguyên hệ thống, hiệu năng cao và hỗ trợ tốt cho việc phát triển các công cụ bảo mật chuyên nghiệp.

Các thành phần hỗ trợ bảo mật trong C++

OpenSSL

- Thư viện mã hóa phổ biến nhất cho C/C++.
- Hỗ trợ:
 - AES
 - DES
 - RSA
 - SHA-256
 - Chữ ký số
 - SSL/TLS

Crypto++

- Thư viện mã hóa mã nguồn mở dành riêng cho C++.
- Hỗ trợ nhiều thuật toán:
 - RSA
 - ElGamal
 - ECC
 - AES
 - Blowfish
 - Twofish

Botan

- Thư viện bảo mật hiện đại cho C++.
- Hỗ trợ:
 - Mã hóa
 - Chữ ký số
 - Xác thực
 - TLS

Libsodium

- Thư viện bảo mật tập trung vào tính đơn giản và an toàn.
- Hỗ trợ:
 - Mã hóa đối xứng
 - Mã hóa bất đối xứng
 - Băm dữ liệu
 - Chữ ký số

Các loại kỹ thuật bảo mật hỗ trợ bởi C++

Loại bảo mật	Mô tả	Thư viện hỗ trợ
Mã hóa đối xứng	Mã hóa và giải mã dùng chung khóa	OpenSSL, Crypto++
Mã hóa bất đối xứng	Sử dụng cặp khóa công khai và bí mật	RSA, ElGamal, ECC
Hàm băm	Kiểm tra tính toàn vẹn dữ liệu	SHA-256, SHA-512
Chữ ký số	Xác thực dữ liệu và nguồn gốc	OpenSSL, Botan
SSL/TLS	Bảo mật truyền thông mạng	OpenSSL

Các công cụ bảo mật nổi bật viết bằng C++

Công cụ	Chức năng
Wireshark	Phân tích gói tin mạng
Nmap	Quét mạng và phát hiện dịch vụ
Snort	Hệ thống phát hiện xâm nhập
ClamAV	Phần mềm diệt virus mã nguồn mở
VeraCrypt	Mã hóa ổ đĩa và dữ liệu
John the Ripper	Kiểm thử độ mạnh mật khẩu

Một số ứng dụng của C++ trong bảo mật

Ứng dụng	Mô tả
Mã hóa dữ liệu	Mã hóa tệp tin và cơ sở dữ liệu
Chữ ký số	Xác thực văn bản điện tử
Bảo mật mạng	Xây dựng Firewall, IDS/IPS
Phân tích gói tin	Giám sát và phát hiện tấn công mạng
Công cụ Pentest	Xây dựng trình quét lỗ hổng và công cụ kiểm thử
Phần mềm Antivirus	Phát hiện và loại bỏ mã độc
Hệ thống nhúng bảo mật	Bảo vệ thiết bị IoT và thiết bị công nghiệp

2.3.2.2. Ngôn ngữ lập trình Python

a. Ngôn ngữ python

Python là một ngôn ngữ lập trình thông dịch, bậc cao, đa mục đích, được phát triển bởi Guido van Rossum và ra mắt lần đầu vào năm 1991.

Với khẩu hiệu: “Simple is better than complex”, Python được đánh giá là dễ học, dễ sử dụng và cực kỳ linh hoạt, trở thành ngôn ngữ lập trình phổ biến hàng đầu trong nhiều lĩnh vực: từ phát triển web, trí tuệ nhân tạo đến an toàn và bảo mật thông tin.

b. Các đặc điểm chính của Python

Đơn giản và dễ học (Simplicity & Readability):

- Cú pháp ngắn gọn, giống tiếng Anh, dễ đọc và dễ hiểu.
- Phù hợp với người mới học lập trình hoặc cần viết script nhanh.

Đa nền tảng (Cross-platform):

- Python chạy được trên nhiều hệ điều hành: Windows, Linux, macOS, Android, Raspberry Pi...
- Không cần biên dịch → thực thi trực tiếp bằng Python interpreter (python, python3).

Thông dịch và động (Interpreted & Dynamically Typed):

- Không cần biên dịch trước khi chạy, dễ debug.
- Kiểu dữ liệu được xác định trong quá trình thực thi, giúp phát triển nhanh chóng.

Hỗ trợ nhiều mô hình lập trình:

- Thủ tục (procedural)

- Hướng đối tượng (object-oriented)
- Hàm (functional)

Tự động quản lý bộ nhớ:

- Python có garbage collector để thu hồi bộ nhớ không dùng đến.
- Giảm lỗi rò rỉ bộ nhớ, giúp lập trình viên tập trung vào logic.

Dễ tích hợp với hệ thống & ngôn ngữ khác:

- Tích hợp tốt với C/C++, Java, .NET, Bash...
- Dễ giao tiếp với hệ điều hành (OS, file system, process...).

c. Python trong an toàn và bảo mật thông tin

Python là một trong những ngôn ngữ phổ biến nhất trong lĩnh vực bảo mật, được dùng rộng rãi bởi: Pentester (kiểm thử bảo mật), Malware Analyst (phân tích mã độc), Blue Team / SOC Analyst (giám sát và phản ứng), Nhà nghiên cứu bảo mật (security researcher) ...

Các thành phần và thư viện bảo mật trong Python

Mã hóa – Thư viện mã hóa mạnh mẽ:

Loại bảo mật	Mô tả	Thư viện hỗ trợ
Mã hóa đối xứng	Dùng chung một khóa để mã hóa/giải mã	cryptography, PyCrypto, pycryptodome

Mã hóa bất đối xứng	Dùng cặp khóa công khai và khóa riêng	cryptography, rsa, PyNaCl
Băm (Hashing)	Tạo mã hóa một chiều không thể đảo ngược	hashlib, bcrypt, argon2
HMAC	Băm có khóa, xác minh tính toàn vẹn thông điệp	hmac, cryptography
Chữ ký số	Xác thực nguồn gốc và tính toàn vẹn dữ liệu	cryptography, PyNaCl

Phân tích lưu lượng mạng (Packet & Traffic Analysis):

- Scapy – Tạo, phân tích, gửi, bắt gói tin mạng
- PyShark – Phân tích file PCAP với Wireshark engine
- Socket, dpkt – Thư viện cấp thấp xử lý gói tin

Phân tích mã độc (Malware Analysis):

- Pefile – Phân tích cấu trúc file PE trên Windows
- Yara-python – Tìm kiếm mẫu trong mã độc
- Volatility – Phân tích bộ nhớ RAM
- Capstone – Giải mã mã máy (disassembler)

Kiểm thử xâm nhập & tấn công (Pentesting & Exploit):

- Nmap – Tích hợp qua Python để quét mạng
- Paramiko – SSH client dùng để brute-force hoặc quản lý máy từ xa
- Requests – Gửi HTTP/HTTPS request (XSS, sqli test...)
- Socket, telnetlib, ftplib – Giao tiếp giao thức mạng

Xác thực & bảo vệ ứng dụng:

- Xác thực bằng mật khẩu, OTP, OAuth2, JWT...
- Xây dựng REST API bảo mật bằng Flask, FastAPI, kết hợp với PyJWT, bcrypt, ssl.

Tự động hóa bảo mật & SOC:

- Python để viết script tự động hóa:
 - + Quét log bất thường
 - + Phân tích syslog
 - + Gửi cảnh báo email
 - + Kết nối với hệ thống SIEM (Splunk, ELK...)

Các công cụ bảo mật nổi bật viết bằng Python

Công cụ	Chức năng
Scapy	Giao thức mạng và packet crafting
Volatility	Phân tích bộ nhớ
TheHarvester	Thu thập OSINT (email, domain, host...)

Recon-ng	Khung OSINT tự động
Impacket	Giao tiếp giao thức Windows: SMB, NTLM...
W3af, sqlmap	Tấn công web: SQLi, XSS...
AutoSploit, PayloadAllTheThings	Pentest tự động và tập hợp khai thác

Một số ứng dụng của Python trong bảo mật

Ứng dụng	Mô tả
Mã hóa dữ liệu	Mã hóa file, mật khẩu, chuỗi thông tin nhạy cảm
Phát hiện tấn công	Phân tích log hệ thống, cảnh báo sự cố
Kiểm thử bảo mật	Quét port, brute-force mật khẩu, khai thác lỗ hổng
Phân tích mã độc	Giải mã shellcode, unpack mã độc

Bảo vệ ứng dụng	Xác thực người dùng, mã hóa JWT, chống CSRF/XSS
Tự động hóa bảo mật	Tự động gửi cảnh báo, cập nhật firewall, phân tích nhật ký
Giao tiếp với hệ thống	Tự động backup, kiểm tra dịch vụ đang chạy, kiểm tra tiến trình

2.3.3. Cài đặt thuật toán, giao diện chương trình

2.3.3.1. Thuật toán

* Giai đoạn Tạo Khóa:

- Chọn một số nguyên tố q đủ lớn.

(Ví dụ: $q = 971$)

- Chọn một số nguyên a sao cho a là căn nguyên thủy modulo q , thỏa mãn:

+ $a < q$

+ a và q nguyên tố cùng nhau ($\gcd(a, q) = 1$)

+ Bậc của a modulo q là $q - 1$

- Chọn một số nguyên bí mật X_A sao cho:

$$1 < X_A < q - 1$$

- Tính giá trị Y_A theo công thức:

$$Y_A = a_A^{X_A} \bmod q$$

- Xác định khóa:

+ Khóa công khai: $\{q, a, Y_A\}$

+ Khóa bí mật: $\{X_A\}$

* Giai đoạn Mã Hóa:

- Giả sử thông điệp ban đầu là M , thỏa mãn:

$$0 < M < q$$

- Chọn ngẫu nhiên một số k sao cho:

$$1 < k < q - 1$$

- Tính thành phần thứ nhất của bản mã:

$$C_1 = a^k \bmod q$$

- Tính giá trị trung gian:

$$K = Y_A^k \bmod q$$

- Tính thành phần thứ hai của bản mã:

$$C_2 = K * M \bmod q$$

- Bản mã được tạo ra là:

$$\{C_1, C_2\}$$

* Giai đoạn Giải mã:

- Nhận bản mã gồm hai thành phần: $\{C_1, C_2\}$

- Sử dụng khóa bí mật X_A để tính:

$$S = C_1^{X_A} \bmod q$$

- Tính nghịch đảo modular của S theo modulo q :

$$S^1 = S^{-1} \bmod q$$

- Tính lại thông điệp gốc M :

$$M = C_2 * S^1 \bmod q$$

- Kết quả M là thông điệp ban đầu đã được giải mã thành công.

2.3.3.2. Giao diện chương trình

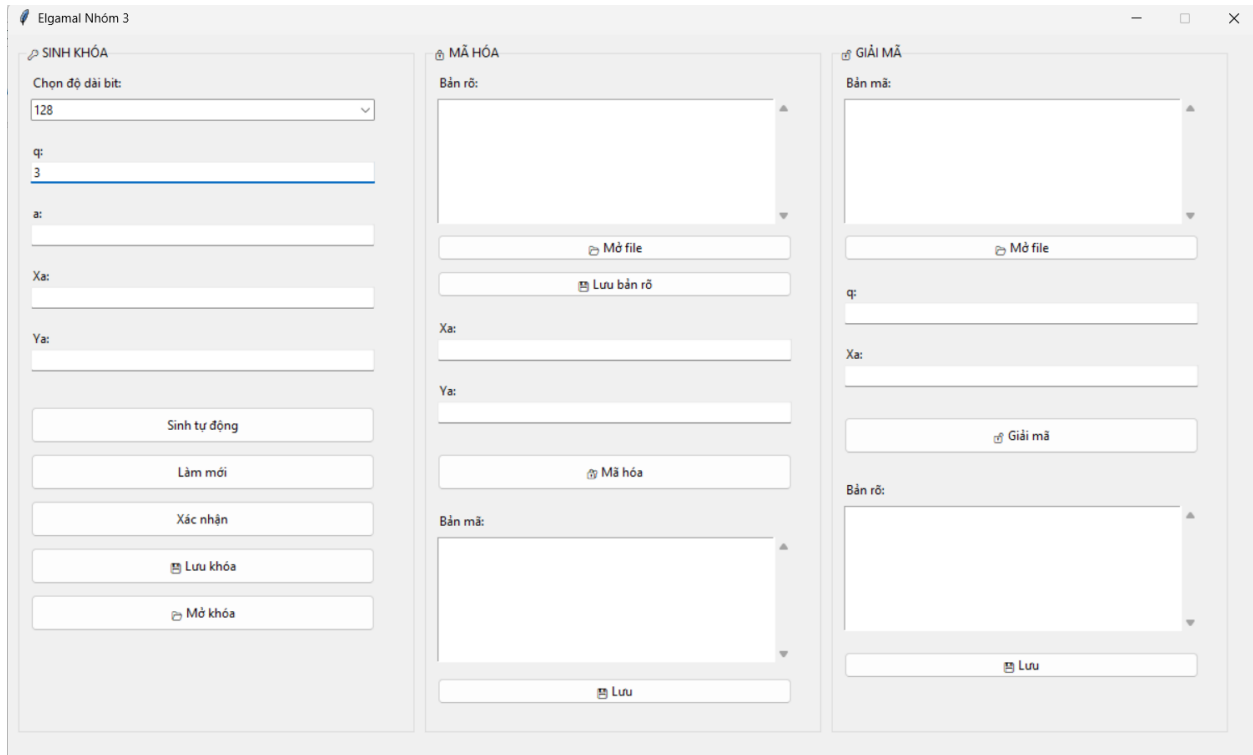
Giao diện chương trình mã hóa và giải mã Elgamal viết bằng ngôn ngữ C++ và Python.

2.3.3.2.1. Giao diện chương trình bằng ngôn ngữ Python

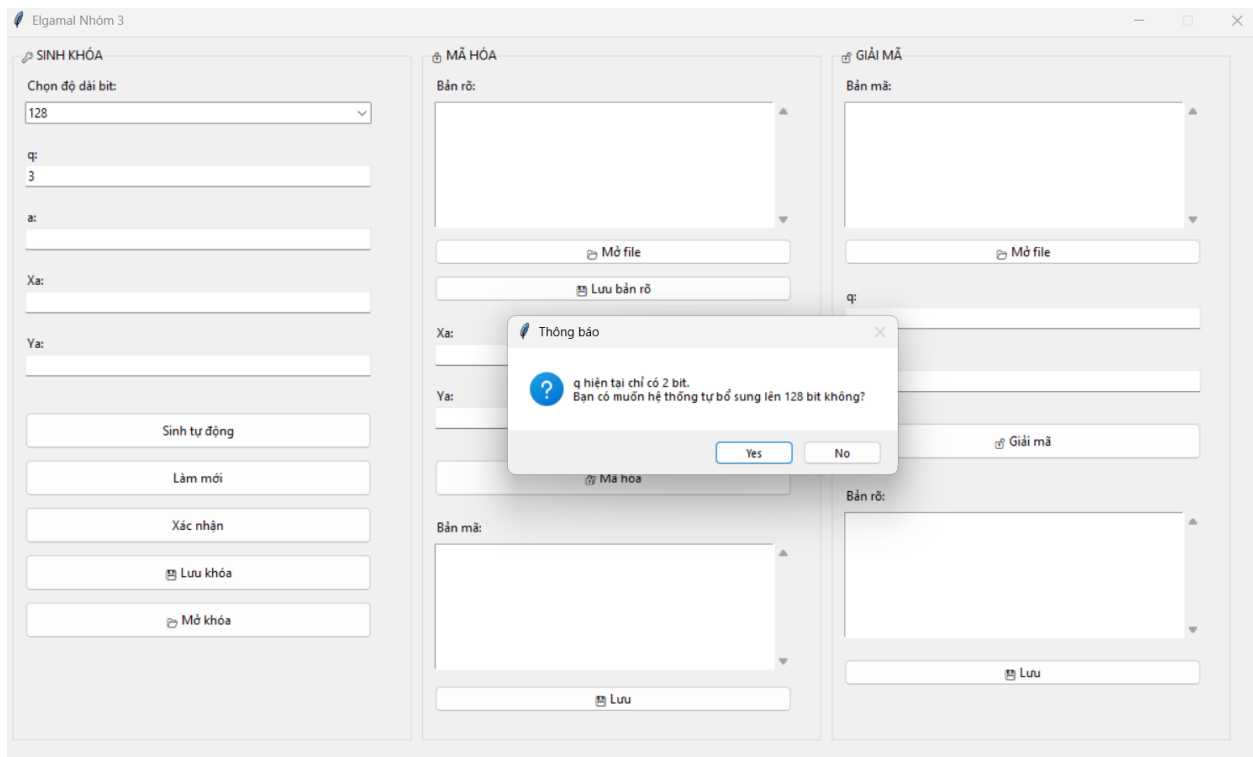
Hình 2.1: Giao diện chương trình Python

Bước 1: Người dùng có thể sinh khóa bằng cách tự động hoặc thủ công:

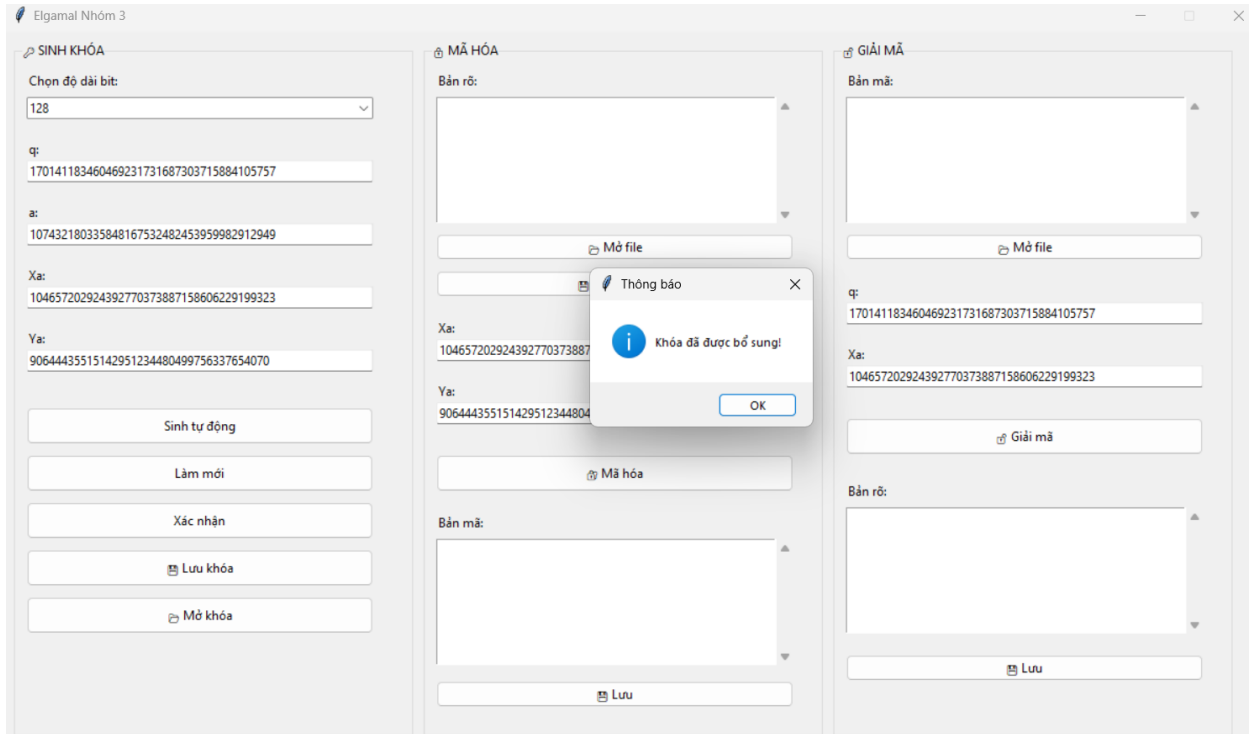
- Sinh khóa thủ công: Người dùng chọn độ dài bit số q mong muốn (128 bits, 256 bits, 512 bits, ...). Rồi nhập một số nguyên tố q bất kỳ có độ dài bit tương ứng và ấn nút “Xác nhận”. Nếu số q nhập vào có độ dài bit chưa đủ hệ thống sẽ tự động bổ sung. Nếu q hợp lệ hệ thống sẽ tự động tính toán các tham số và khóa rồi hiển thị lên màn hình.



Hình 2.2: Nhập thủ công số q (Python)

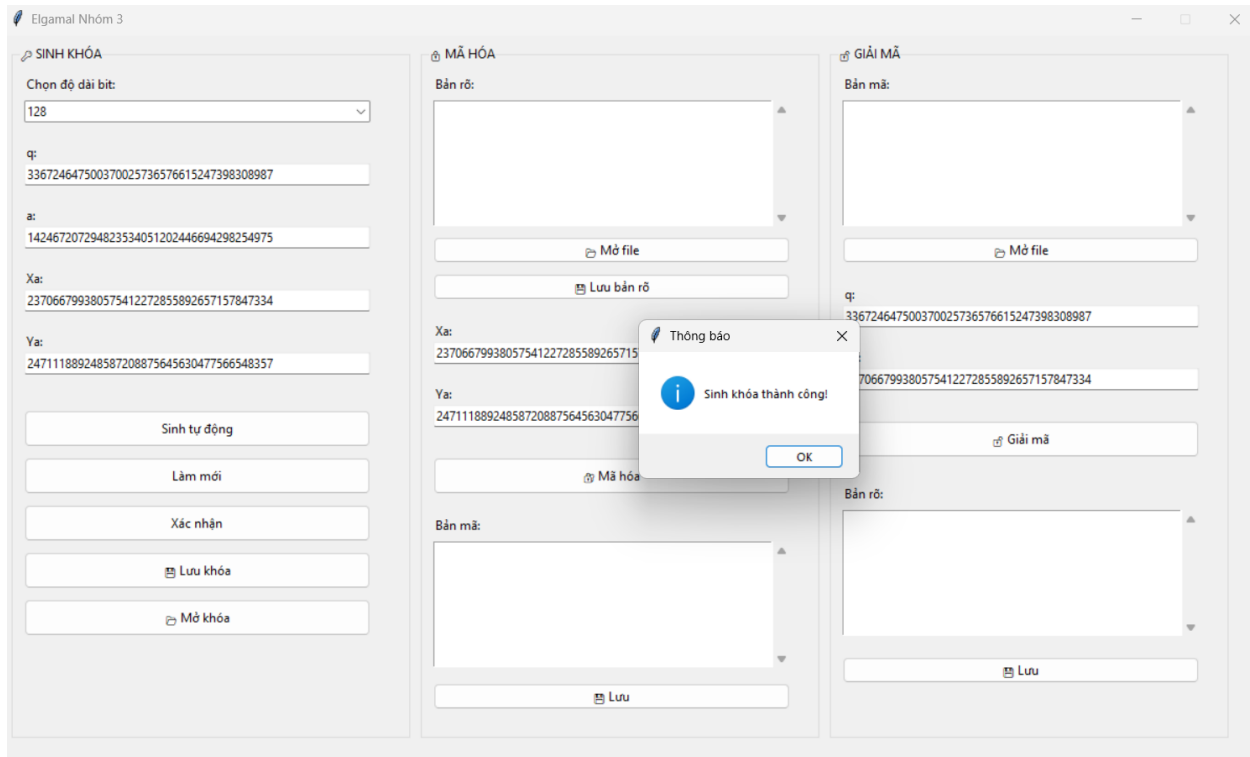


Hình 2.3: Thông báo chọn tự bổ sung q (Python)



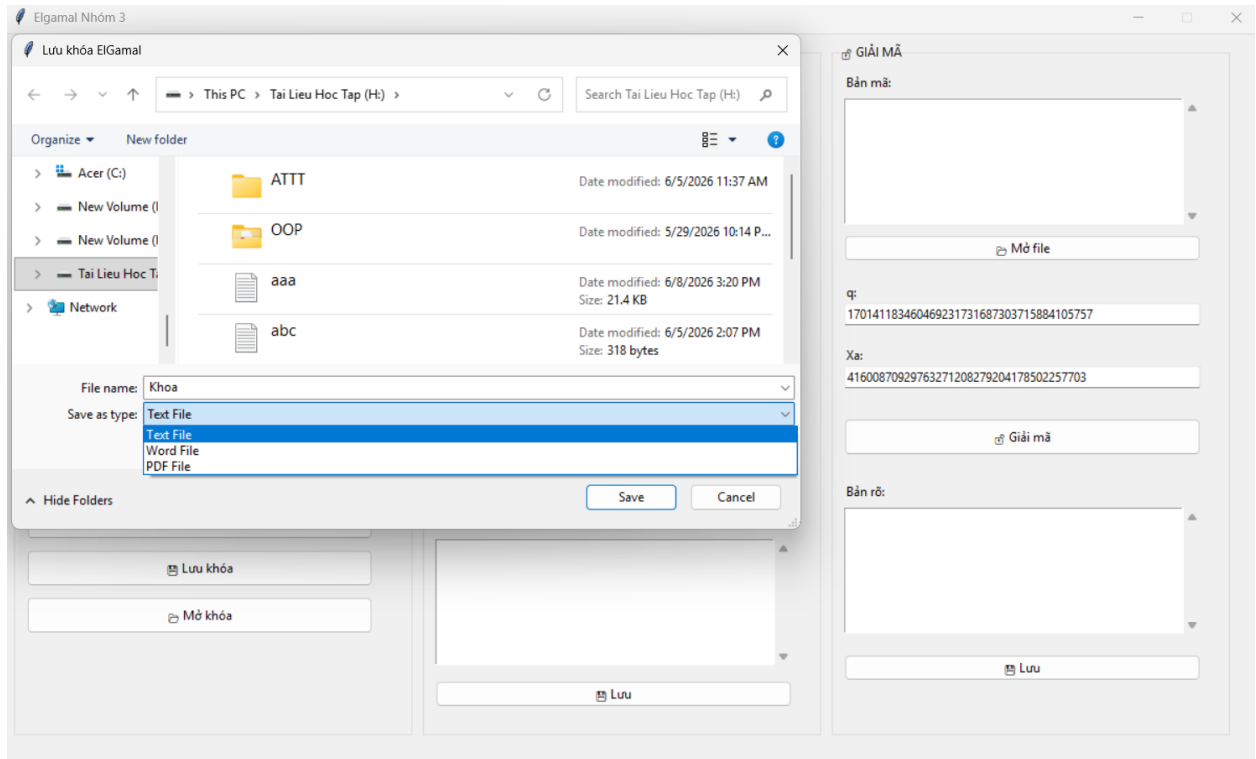
Hình 2.4: Hệ thống tự bổ sung q (Python)

- Sinh khóa tự động : Người dùng ấn vào nút “Sinh tự động”. Hệ thống sẽ chọn ngẫu nhiên một số q bất kỳ và sẽ tự tính toán và hiển thị a , Xa và Ya .

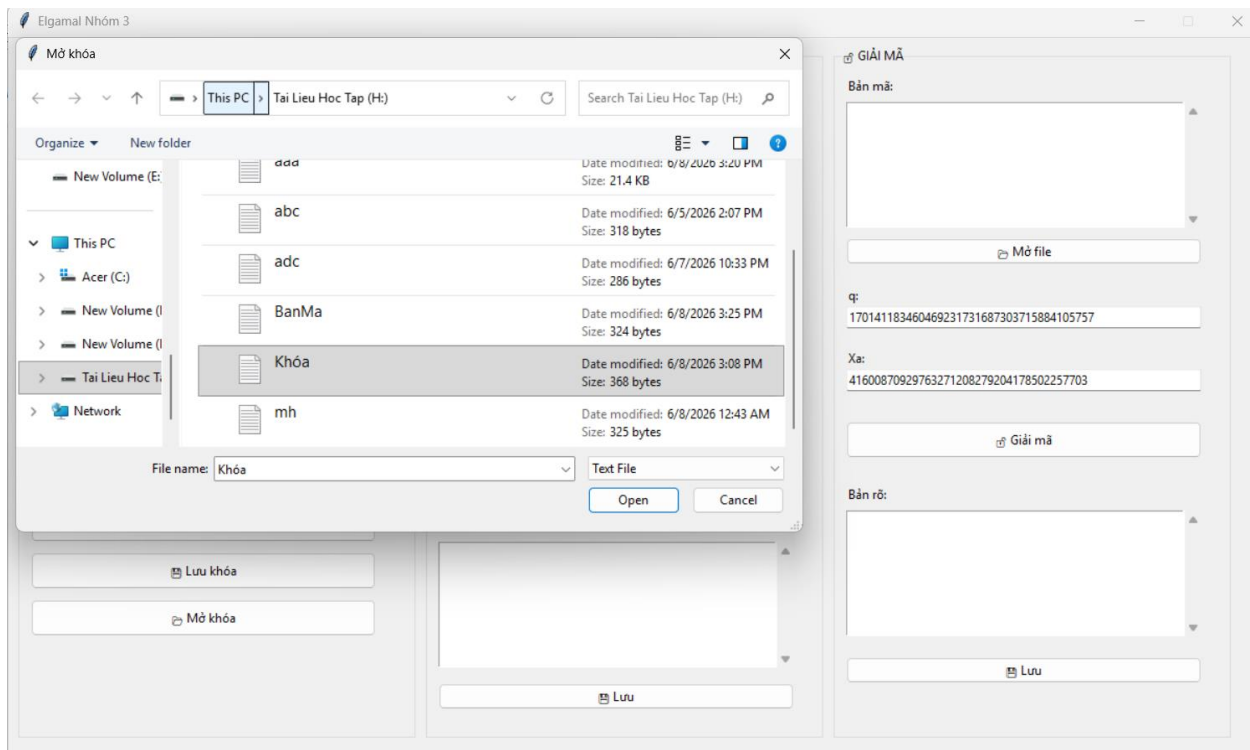


Hình 2.5: Sinh khóa tự động(Python)

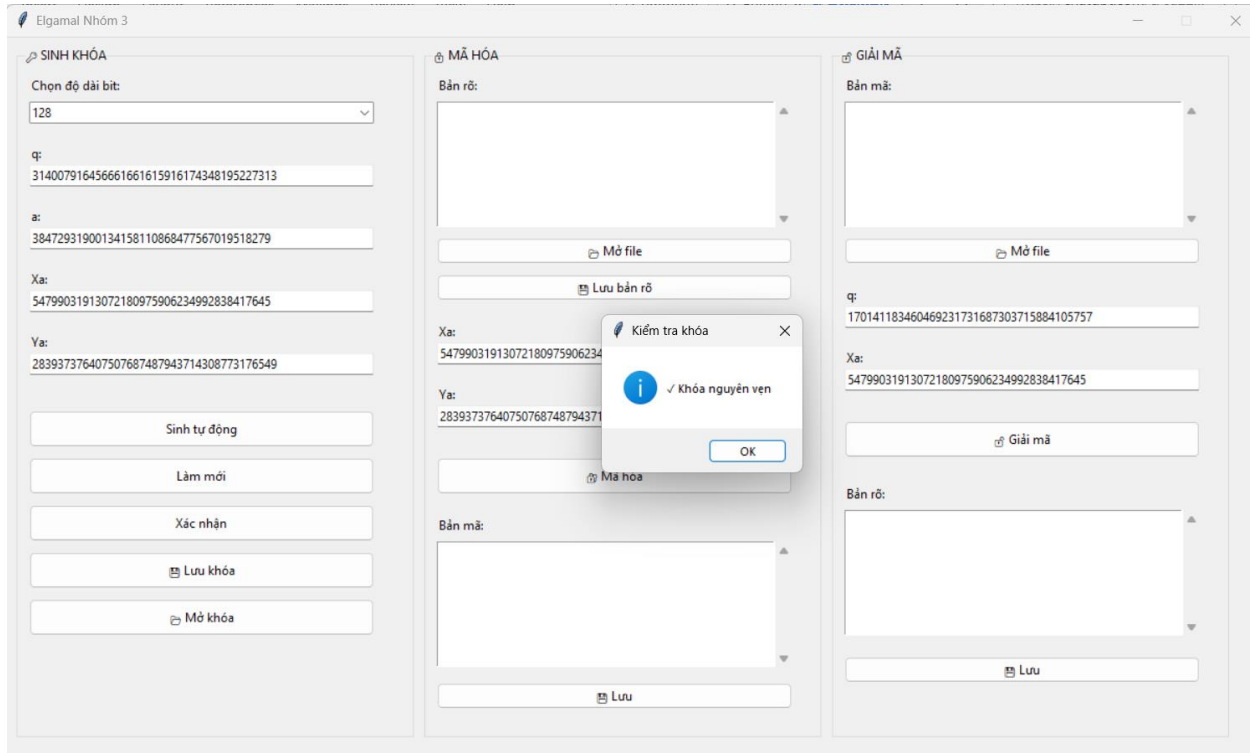
- Hệ thống sẽ hiển thị lại và lưu lại các giá trị của các biến q, a, X_a và Y_a vào một file và người dùng cũng có thể lưu và mở file khóa đã tạo thủ công và khi mở lại hệ thống sẽ kiểm tra và thông báo khóa còn toàn vẹn không.



Hình 2.6: Lưu file khóa(Python)

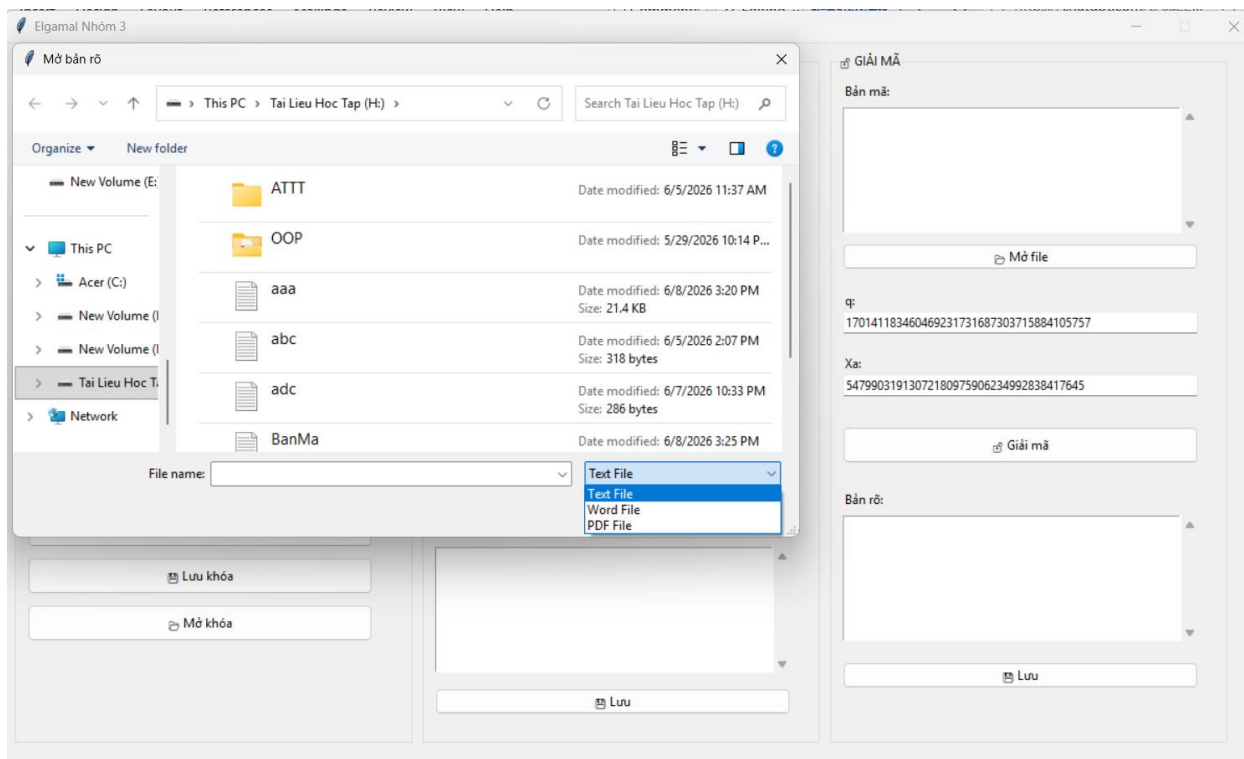


Hình 2.7: Mở file khóa(Python)

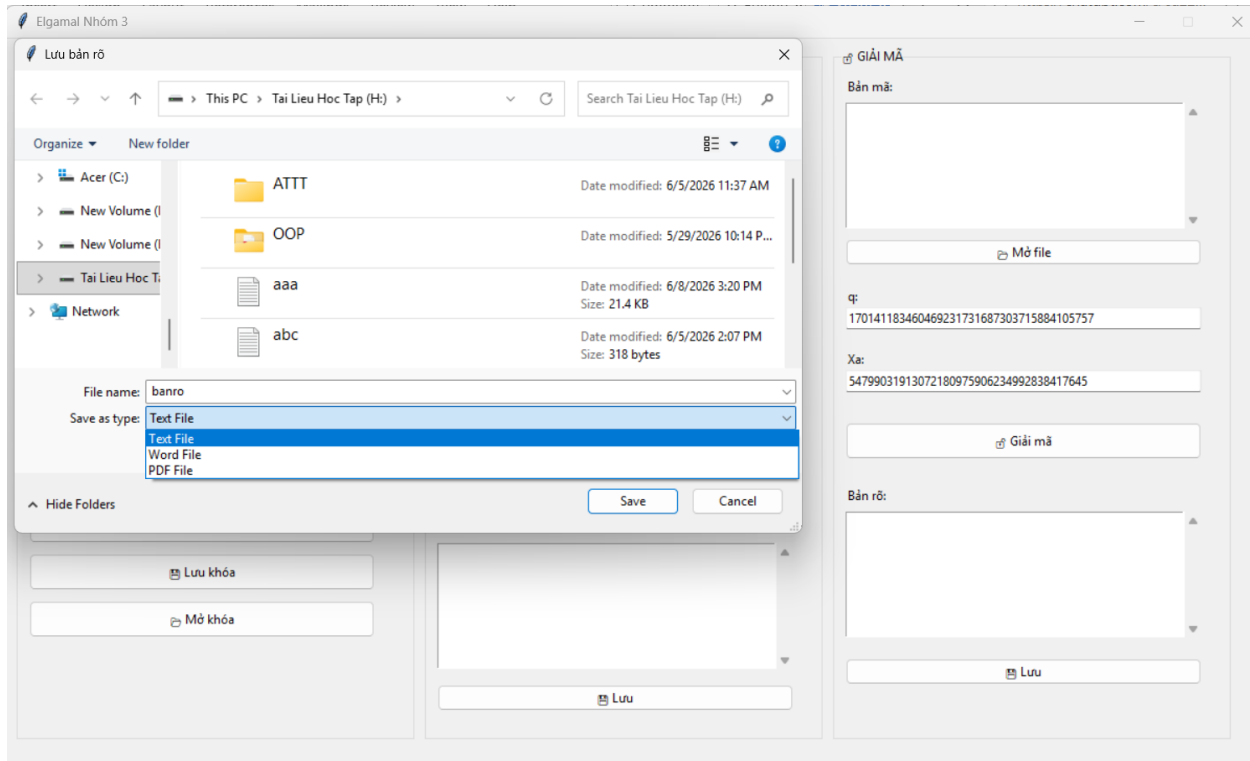


Hình 2.8: Hệ thống kiểm tra và thông báo tính toàn vẹn của khóa(Python)

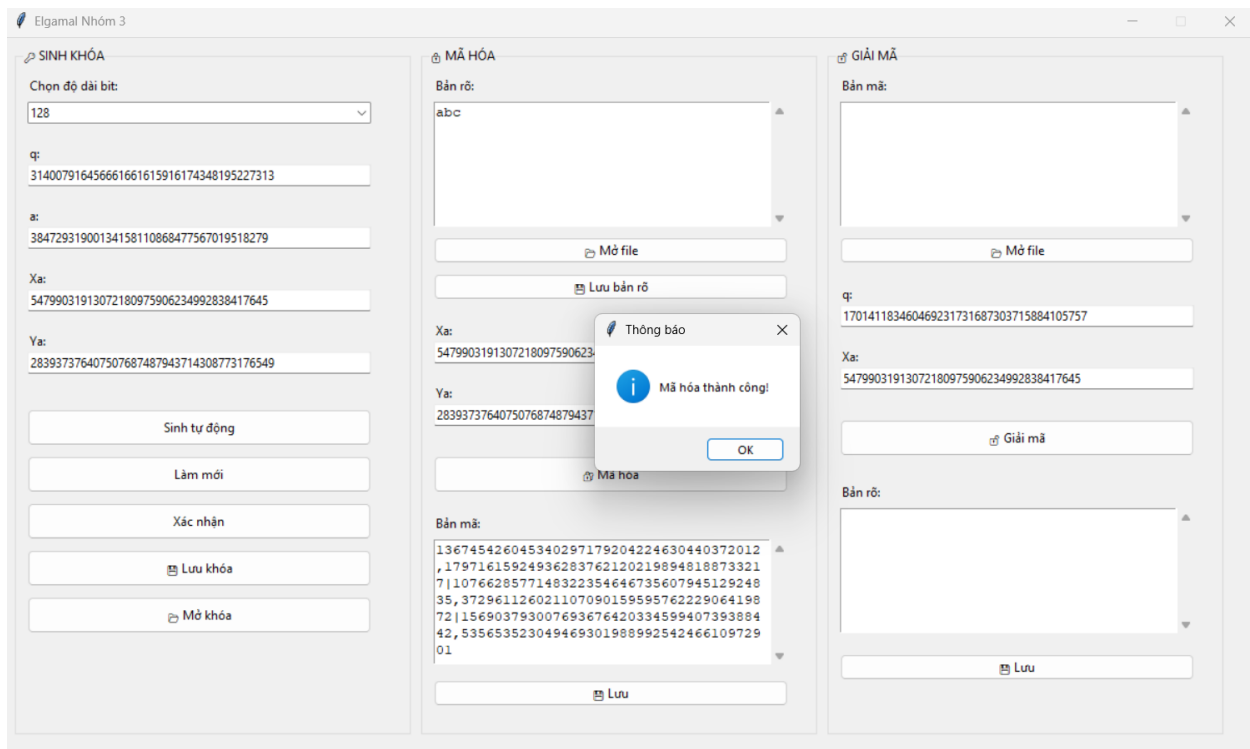
Bước 2: Người dùng có thể nhập thủ công dữ liệu cần mã hóa vào vùng nhập hoặc có thể mở file docx, txt, pdf, ... để mã hóa và có thể lưu văn bản bản rõ nhập thủ công. Sau khi mã hóa, hệ thống hiển thị thông báo “Mã hóa thành công”.



Hình 2.9: Mở file để mã hóa(Python)

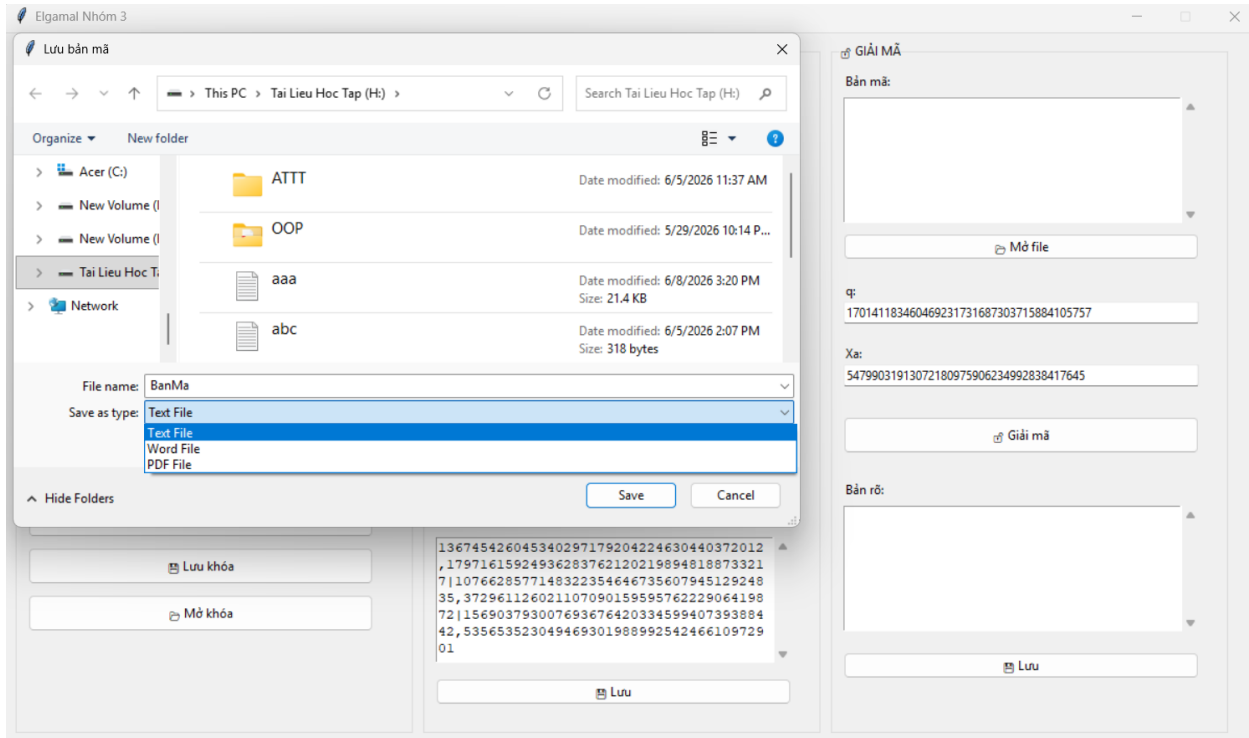


Hình 2.10: Lưu bản rõ nhập thủ công(Python)



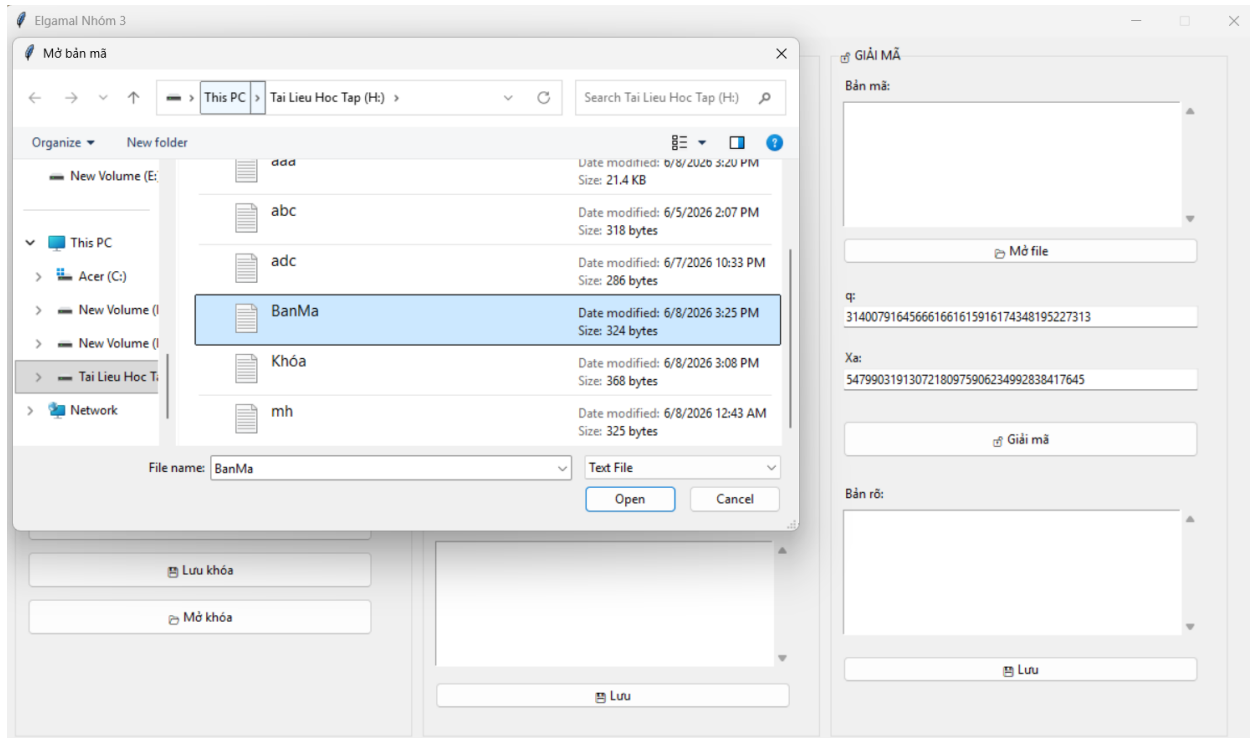
Hình 2.11: Thông báo mã hóa thành công(Python)

Người dùng nhấn nút “Lưu” để lưu file đã được mã hóa.

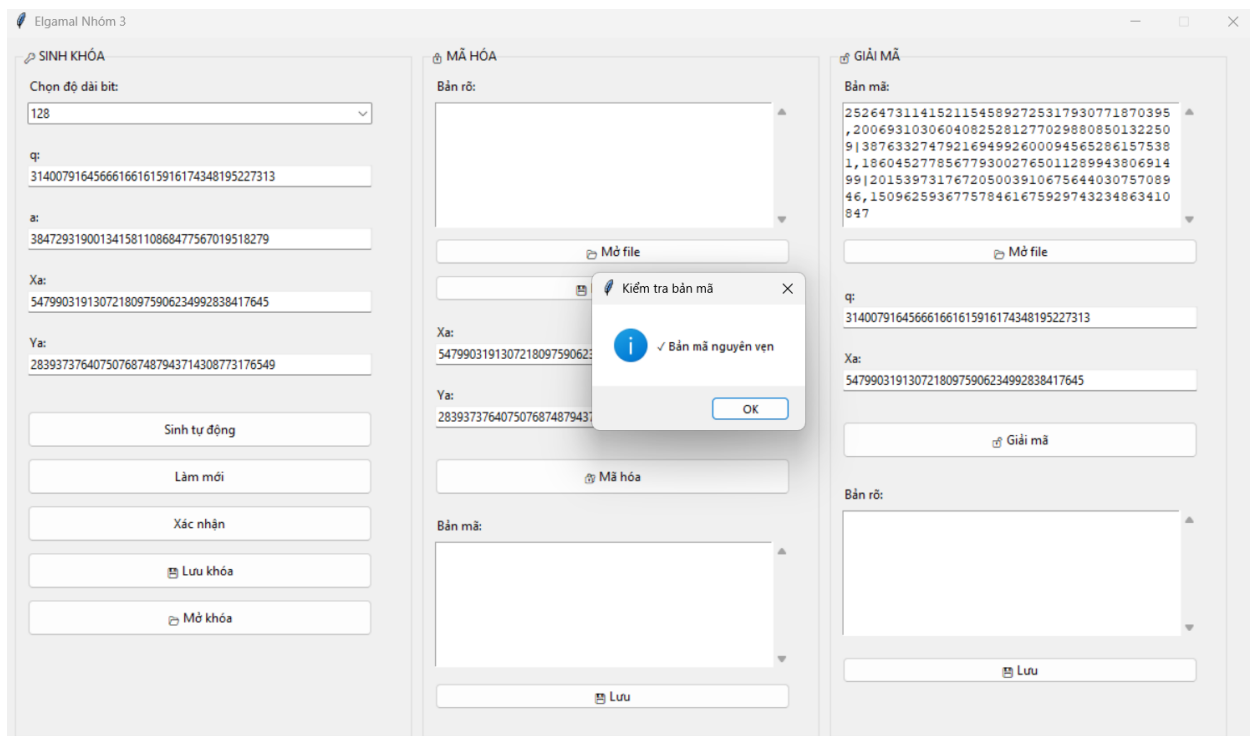


Hình 2.12: Lưu file bản mã vừa được mã hóa (Python)

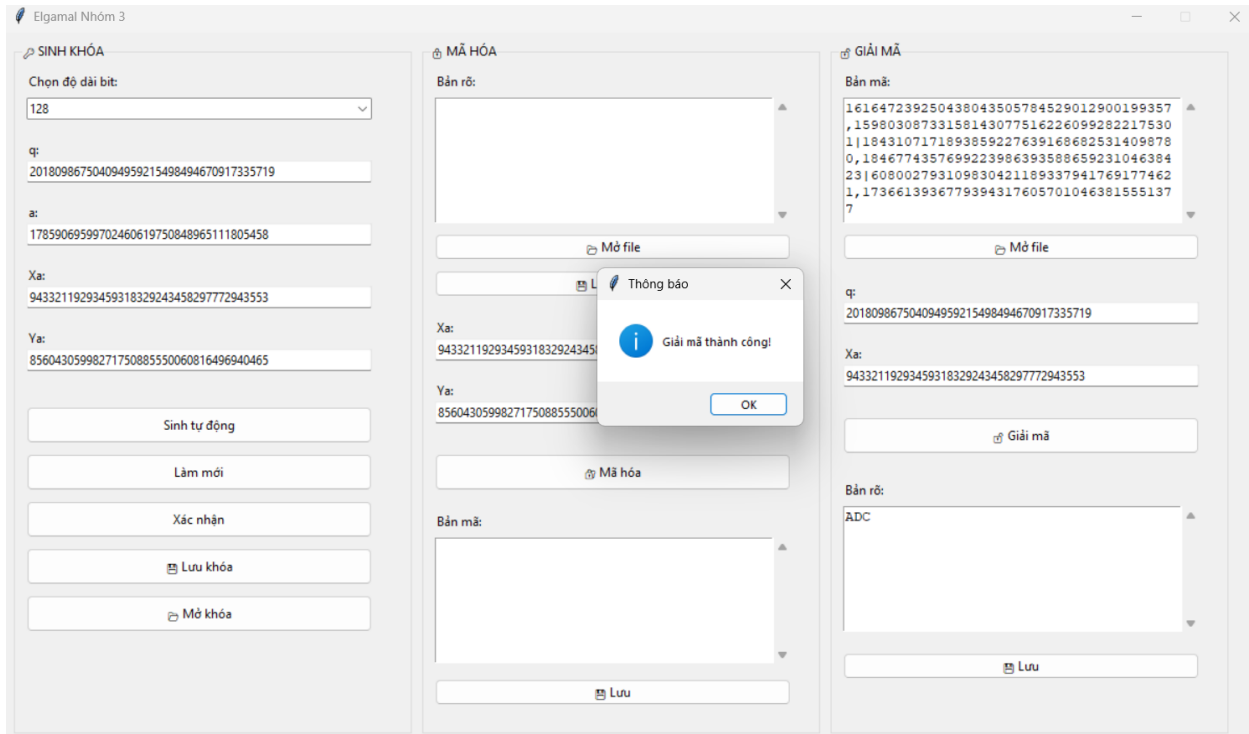
Bước 3: Người dùng mở file đã được mã hóa, khi mở hệ thống sẽ kiểm tra và thông báo tính toàn vẹn của file và dùng các giá trị biến q và Xa đã được sinh khóa ở bước 1 mà người dùng đã lưu lại để giải mã. Hệ thống sẽ hiển thị thông báo “Giải mã thành công” nếu file khóa và file bản mã còn nguyên vẹn.



Hình 2.13: Mở file bản mã (Python)

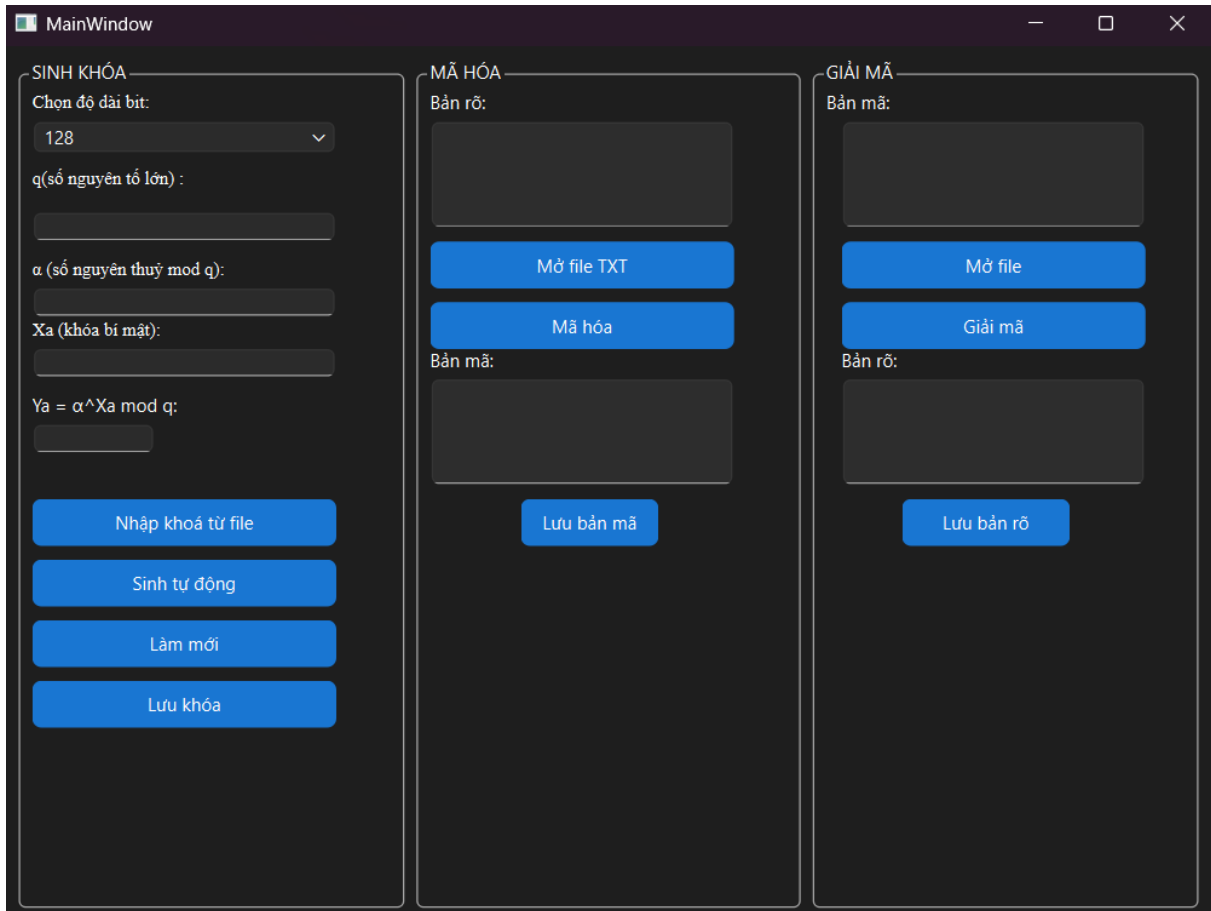


Hình 2.14: Hệ thống kiểm tra và thông báo tính toàn vẹn của bản mã (Python)



Hình 2.15: Thông báo giải mã thành công (Python)

2.3.3.2.2. Giao diện chương trình bằng ngôn ngữ C++



Hình 2.16 : Giao diện chương trình C++

Bước 1: Người dùng có thể sinh khóa bằng cách tự động hoặc thủ công

- Sinh khóa tự động: Người dùng lựa chọn độ dài bit mong muốn của số nguyên tố q , sau đó nhấn nút “Sinh tự động”. Hệ thống sẽ tự động sinh số nguyên tố q , xác định phần tử sinh α , sinh khóa bí mật X_A và tính khóa công khai Y_A . Các tham số và khóa được hiển thị trên giao diện chương trình.

The screenshot shows a software window titled "MainWindow" with a dark theme. It is divided into three vertical panels:

- SINH KHÓA (Key Generation):**
 - Chọn độ dài bit: 128 (dropdown)
 - q (số nguyên tố lớn): 2203 (text input)
 - α (số nguyên thủy mod q): 5 (text input)
 - Xa (khóa bí mật): 1138 (text input)
 - $Y_a = \alpha^X a \text{ mod } q$: 498 (text input)
 - Buttons: Nhập khoá từ file, Sinh tự động, Làm mới, Lưu khoá
- MÃ HÓA (Encryption):**
 - Bản rõ: (text input)
 - Button: Mở file TXT
 - Button: Mã hóa
 - Bản mã: (text input)
 - Button: Lưu bản mã
- GIẢI MÃ (Decryption):**
 - Bản mã: (text input)
 - Button: Mở file
 - Button: Giải mã
 - Bản rõ: (text input)
 - Button: Lưu bản rõ

Hình 2.17: Sinh khóa tự động

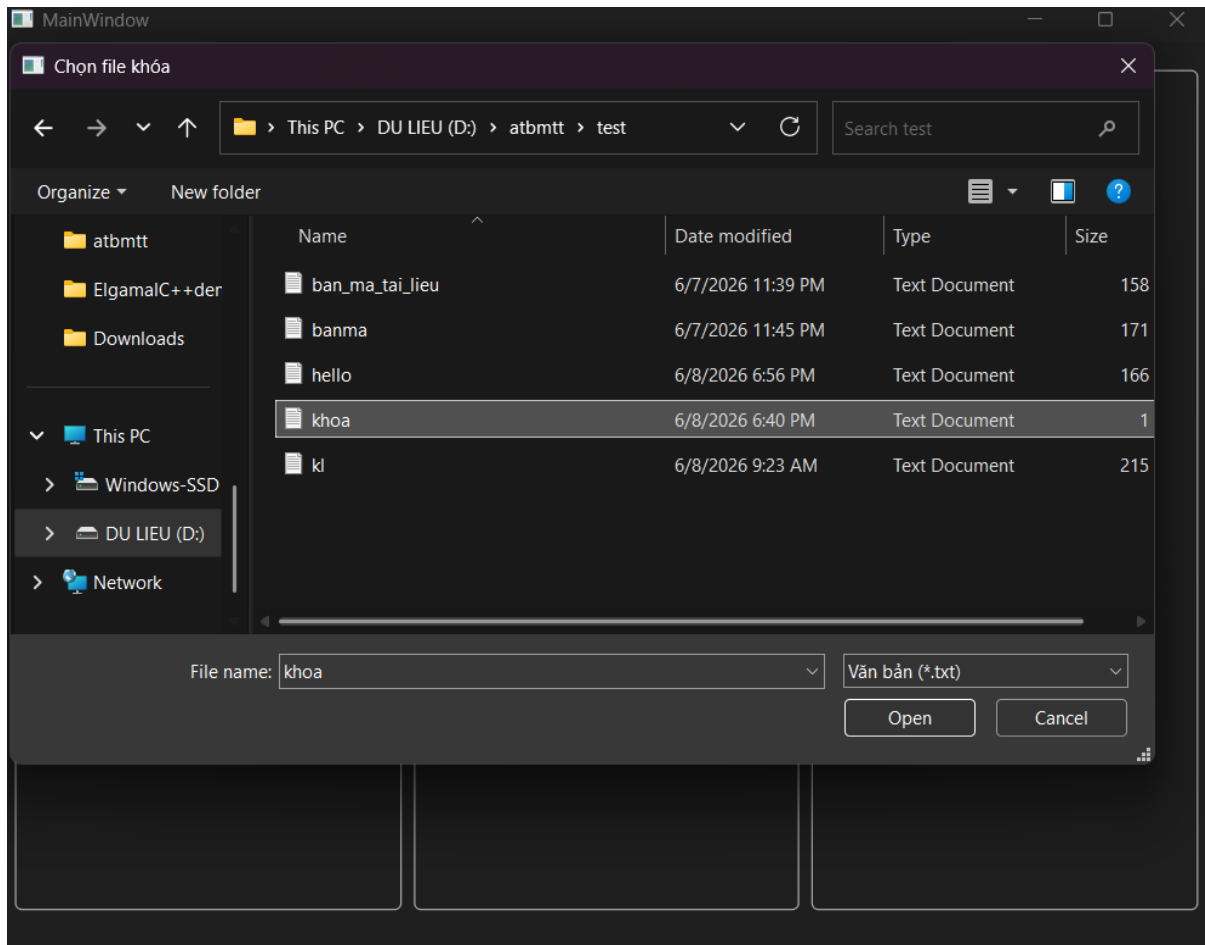
- Sinh khóa thủ công: Người dùng nhập các tham số q , α và khóa bí mật X_A . Sau khi thực hiện lệnh sinh khóa, hệ thống tự động tính giá trị khóa công khai Y_A và hiển thị kết quả trên giao diện.

The screenshot shows a software window titled 'MainWindow' with three main functional panels:

- SINH KHÓA (Key Generation):**
 - 'Chọn độ dài bit:' (Choose bit length): A dropdown menu showing '128'.
 - 'q(số nguyên tố lớn):' (q, large prime number): A text input field containing '2833'.
 - 'α (số nguyên thủy mod q):' (α, primitive root mod q): A text input field containing '5'.
 - 'Xa (khóa bí mật):' (Xa, secret key): An empty text input field.
 - 'Ya = α^Xa mod q:': An empty text input field.
 - Buttons at the bottom: 'Nhập khóa từ file' (Import key from file), 'Sinh tự động' (Generate automatically), 'Làm mới' (Refresh), and 'Lưu khóa' (Save key).
- MÃ HÓA (Encryption):**
 - 'Bản rõ:' (Plaintext): A large empty text area.
 - 'Mở file TXT' (Open TXT file): A blue button.
 - 'Mã hóa' (Encrypt): A blue button.
 - 'Bản mã:' (Ciphertext): A large empty text area.
 - 'Lưu bản mã' (Save ciphertext): A blue button.
- GIẢI MÃ (Decryption):**
 - 'Bản mã:' (Ciphertext): A large empty text area.
 - 'Mở file' (Open file): A blue button.
 - 'Giải mã' (Decrypt): A blue button.
 - 'Bản rõ:' (Plaintext): A large empty text area.
 - 'Lưu bản rõ' (Save plaintext): A blue button.

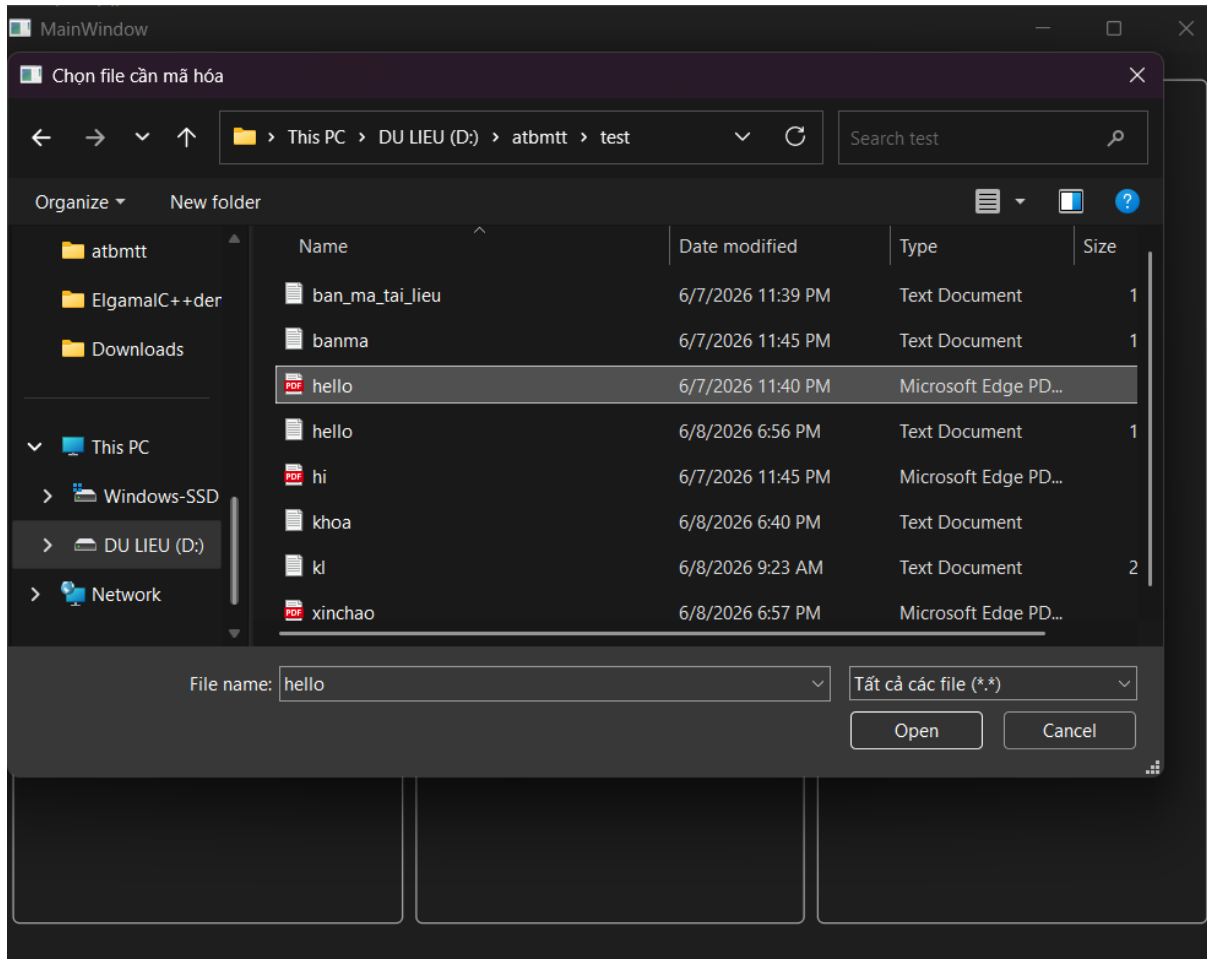
Hình 2.18: Sinh khóa thủ công

- Nạp khóa từ tệp: Người dùng chọn tệp chứa thông tin khóa đã được lưu trước đó. Hệ thống đọc dữ liệu từ tệp, kiểm tra tính toàn vẹn của nội dung và tự động hiển thị các tham số khóa lên giao diện chương trình.



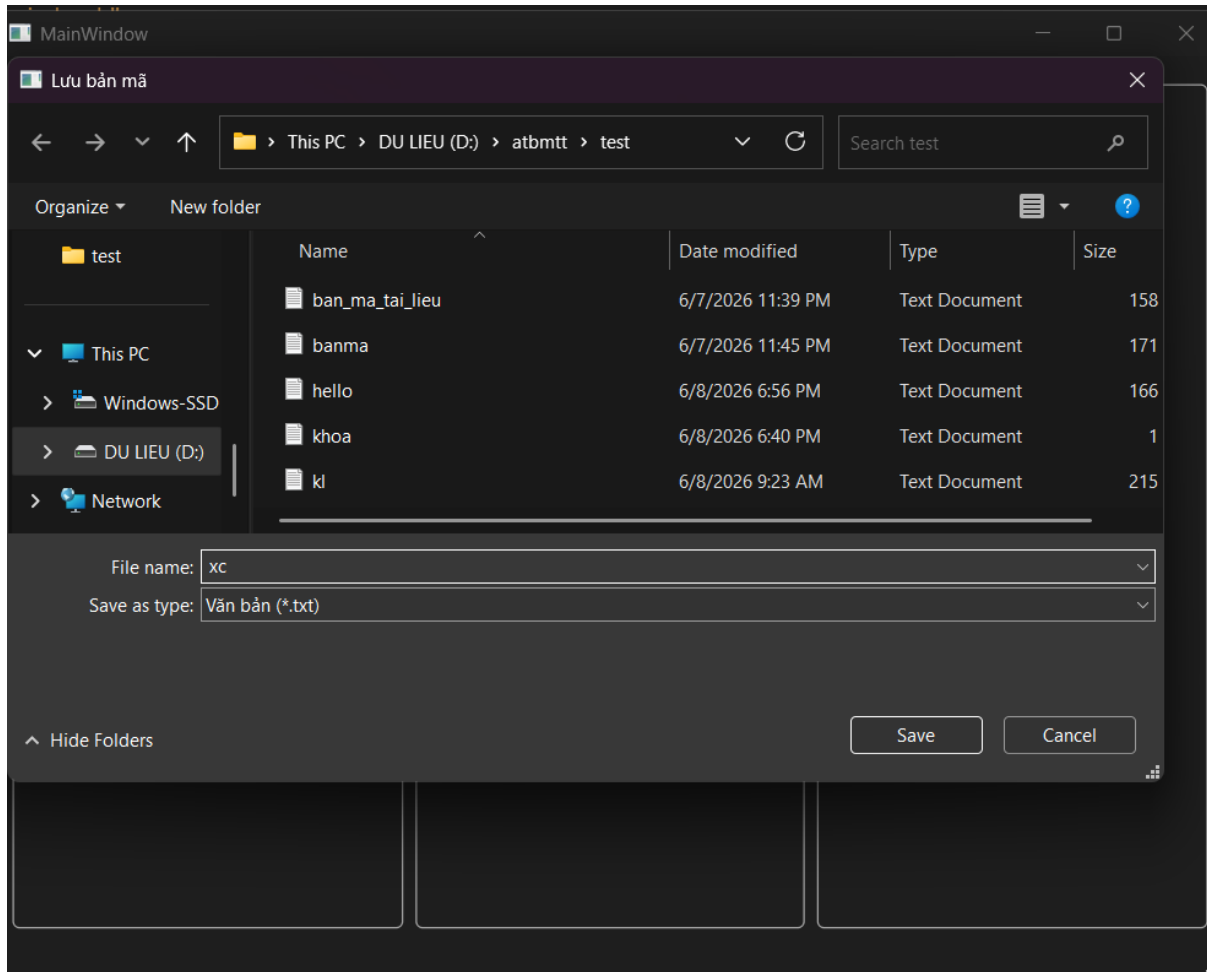
Hình 2.19: Chọn tệp chứa thông tin khóa

Bước 2: Người dùng có thể nhập thủ công dữ liệu cần mã hóa vào vùng nhập hoặc có thể mở file docx, txt, pdf để mã hóa.



Hình2.20: Mở file để mã hoá

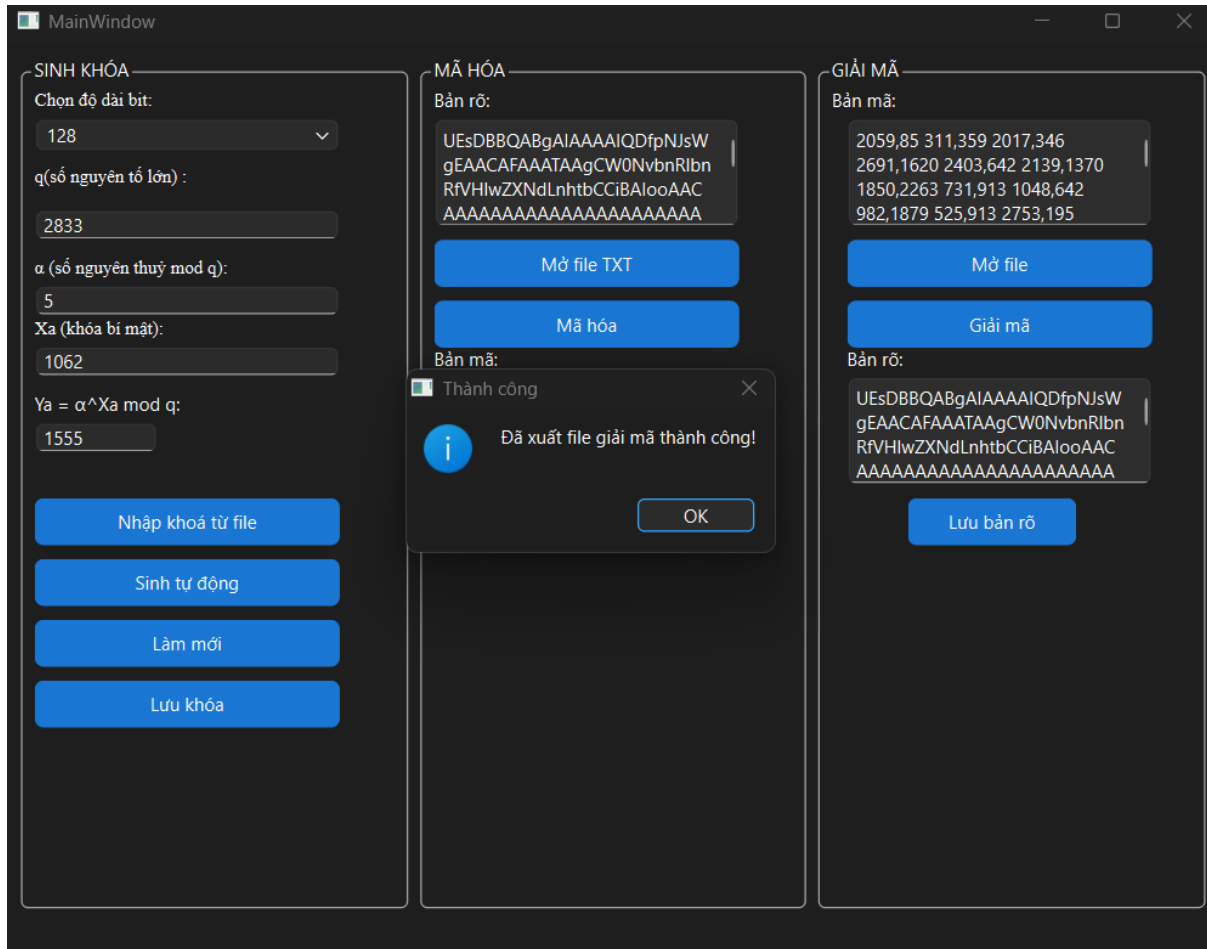
Người dùng nhấn nút “Lưu” để lưu file đã được mã hóa.



Hình 2.21: Lưu file đã dc mã hoá

Bước 3: Người dùng mở file đã được mã hóa và nhập giá trị cho khoá đã được sinh khóa ở bước 1

Người dùng lưu file đã được giải mã (với đuôi đúng định dạng file đã mã hóa)



Hình 2.22: Giải mã thành công

Chương 3: Kết luận và bài học kinh nghiệm

3.1. Kiến thức kỹ năng đã học được trong quá trình thực hiện đề tài.

Thông qua việc nghiên cứu và xây dựng chương trình mã hóa, giải mã hệ mật ElGamal, các thành viên trong nhóm đã rèn luyện và tích lũy được nhiều kiến thức cũng như kỹ năng quan trọng:

- **Kiến thức nền tảng về An toàn thông tin:** Nắm vững cơ sở toán học của các hệ mật mã khóa bất đối xứng, đặc biệt là bài toán logarit rời rạc (Discrete Logarithm Problem). Hiểu rõ quy trình sinh khóa, mã hóa, giải mã cũng như vai trò của ElGamal trong việc xác thực, đảm bảo tính toàn vẹn và bảo mật dữ liệu.
- **Kỹ năng lập trình thực chiến:** Nâng cao tư duy logic và kỹ năng lập trình khi trực tiếp cài đặt các thuật toán toán học phức tạp (như Miller-Rabin, Euclid mở rộng, bình phương và nhân) bằng hai ngôn ngữ là C++ và Python. Quá trình này giúp nhóm thấy rõ được ưu thế linh hoạt, phát triển nhanh của Python cũng như khả năng quản lý bộ nhớ, tối ưu hiệu năng tính toán mạnh mẽ của C++.
- **Kỹ năng phân tích và thiết kế phần mềm:** Củng cố khả năng hệ thống hóa bài toán thông qua việc xây dựng chi tiết các đặc tả Use Case, thiết kế kịch bản hoạt động, giúp luồng xử lý của chương trình bám sát đúng yêu cầu lý thuyết.
- **Kỹ năng mềm:** Hoàn thiện khả năng làm việc nhóm, kỹ năng phân công công việc, giải quyết xung đột khi tích hợp code và kỹ năng soạn thảo tài liệu báo cáo học thuật.

3.2. Bài học kinh nghiệm

Trong quá trình từ lúc tìm hiểu lý thuyết đến khi ra mắt giao diện ứng dụng hoàn chỉnh, nhóm đã rút ra được một số kinh nghiệm quý báu:

- **Kiểm soát chặt chẽ dữ liệu đầu vào:** Trong mật mã học, chỉ một sai sót nhỏ ở tham số đầu vào cũng làm sai lệch toàn bộ kết quả. Điển hình như việc kiểm tra chặt chẽ số q có phải là số nguyên tố hay phân tử a có phải là căn nguyên thủy modulo q hay không. Việc xây dựng các khối xử lý ngoại lệ (Try-Catch) và hiển thị thông báo lỗi rõ ràng trên giao diện là bắt buộc.
- **Xử lý các phép toán với số lớn (Big Integer):** Thuật toán ElGamal yêu cầu tính toán với số mũ rất lớn. Nếu như Python hỗ trợ mặc định cho số nguyên lớn rất tốt, thì với C++, nhóm đã nhận ra sự cần thiết của việc phải hiểu sâu về giới hạn kiểu dữ liệu và cách áp dụng khéo léo thuật toán bình phương và nhân liên tiếp để tránh hiện tượng tràn bộ nhớ (overflow).
- **Quản lý tiến độ dự án:** Việc phân bổ thời gian hợp lý giữa các giai đoạn: nghiên cứu công thức toán học, thiết kế giao diện (UI), cài đặt thuật toán và viết báo cáo tổng kết là chìa khóa để nhóm đi đến kết quả cuối cùng đúng hạn.

3.3. Đề xuất về tính khả thi của chủ đề nghiên cứu, những thuận lợi, khó khăn.

3.3.1. Tính khả thi của đề tài

- Đề tài mang tính khả thi rất cao và đã được nhóm triển khai thành công thành một phần mềm hoạt động ổn định trên cả C++ và Python.

- Giải pháp không chỉ dừng lại ở mức mô phỏng học thuật mà hoàn toàn có thể tiếp tục mở rộng để ứng dụng vào các kịch bản thực tế như: xây dựng module mã hóa file tài liệu văn bản, ứng dụng vào giao thức trao đổi khóa an toàn, hoặc phát triển thành hệ thống chữ ký số dùng trong các phần mềm quản lý, hợp đồng điện tử.

3.3.2. *Thuận lợi*

- Nền tảng lý thuyết rõ ràng: ElGamal là một thuật toán tiêu chuẩn, sở hữu nền tảng toán học logic, chặt chẽ. Hệ thống tài liệu trong nước và quốc tế nghiên cứu về thuật toán này rất đa dạng và chi tiết.
- Cộng đồng và công cụ hỗ trợ mạnh mẽ: Hai ngôn ngữ C++ và Python có cộng đồng lập trình viên khổng lồ cùng nhiều tài liệu hỗ trợ việc thiết kế giao diện (GUI), giúp nhóm nhanh chóng vượt qua các lỗi kỹ thuật trong quá trình code.
- Sự phối hợp nhóm tốt: Các thành viên áp dụng tốt khả năng tổ chức, có sự phân chia rõ ràng giữa người làm giao diện, người cài đặt thuật toán xử lý logic và người viết báo cáo, giúp duy trì năng lượng và tiến độ công việc một cách trơn tru.

3.3.3. *Khó khăn*

- Hiệu năng phân cứng khi xử lý khóa lớn: Đặc thù của mã hóa ElGamal là kích thước bản mã thường lớn gấp đôi bản rõ. Khi nhóm tiến hành chạy thử nghiệm với các độ dài bit lớn (ví dụ trên 1024-bit) để tăng cường bảo mật, hệ thống đòi hỏi tài nguyên tính toán cao và mất nhiều thời gian hơn để giải bài toán sinh số ngẫu nhiên lớn.
- Đồng bộ hóa thuật toán trên đa ngôn ngữ: Việc đảm bảo hàm mã hóa bên phía C++ có thể xuất ra file mà hàm giải mã bên Python vẫn đọc hiểu và giải mã chính xác (và ngược lại) đòi hỏi sự thống nhất cao về mặt định dạng kiểu dữ liệu đầu ra và cách xử lý chuỗi ký tự (encoding).

Tài Liệu Tham Khảo

- [1]. Bùi Doãn Khanh, Nguyễn Đình Thúc, Mã hóa thông tin – Lý thuyết và ứng dụng, NXB Lao động xã hội, 2011.
- [2]. Các tài liệu về hệ mật mã khóa công khai và thuật toán Elgamal trên Internet.